



Année 2024–2025

MASTER 2 Professionnel

Mention : Mathématiques appliquées,
statistique

Spécialité

Ingénierie Mathématique

Rapport de Stage

**Segmentation perceptuelle d'images : combiner modèles
probabilistes et méthodes d'apprentissage**

Étudiant	Kuassi Pierre Dovodji Courriel : dovodji.e2403610@etud.univ-ubs.fr
Référente à l'université	Claire Launay, Maîtresse de conférences Courriel : claire.launay@univ-ubs.fr
Date de la soutenance	27/08/2025

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce mémoire.

*Je remercie tout particulièrement ma directrice de recherche, **Claire Launay**, pour ses conseils avisés, sa patience et son soutien tout au long de ce travail. Sa rigueur scientifique et son expertise ont grandement enrichi cette étude.*

Je souhaite également remercier l'ensemble des enseignants et membres du jury pour leur accompagnement et leurs précieux retours.

Résumé

Ce mémoire explore une approche hybride de la segmentation d'images, combinant modèles probabilistes et méthodes d'apprentissage profond, notamment le modèle SAM. L'objectif est de tirer parti de la robustesse des modèles statistiques pour quantifier l'incertitude, tout en exploitant la puissance de détection des réseaux neuronaux. Les résultats montrent les limites des métriques classiques et mettent en lumière l'intérêt de l'entropie comme indicateur d'évaluation plus fiable. Des pistes d'amélioration sont proposées pour enrichir les outils d'analyse de segmentation.

Abstract

This thesis explores a hybrid approach to image segmentation, combining probabilistic models and deep learning methods, notably the SAM model. The objective is to leverage the robustness of statistical models to quantify uncertainty, while exploiting the detection power of neural networks. The results show the limits of traditional metrics and highlight the interest of entropy as a more reliable evaluation indicator. Avenues for improvement are proposed to enrich segmentation analysis tools.

Mots-clés : Segmentation d'images, Métriques d'évaluation, F-score, ARI, Masques de segmentation, Qualité perceptuelle, SAM, FlexMM, Algorithme

Keywords : Image segmentation, Evaluation metrics, F-score, ARI, Segmentation masks, Perceptual quality, SAM, FlexMM, Algorithm

Table des matières

Introduction	8
1 Cadre général	10
1.1 Contexte	10
1.2 Segmentation d'images	11
1.2.1 Définition de la segmentation d'images	11
1.2.2 Objectifs de la segmentation d'images	12
1.3 État d'art de la segmentation	13
1.3.1 Segmentation de région non texturées (SRNT)	13
1.3.2 Segmentation basée sur les contours	13
1.3.3 Segmentation par classification ou seuillage	14
1.3.4 Segmentation basée sur l'apprentissage automatique	14
1.4 Métriques pour l'évaluation	15
1.4.1 La matrice de confusion	15
1.4.2 Précision	15
1.4.3 Rappel	15
1.4.4 F-Score F	16
1.4.5 Indice de Rand ajusté (ARI)	16
2 Présentation des articles de référence	17
2.1 Modèles de mélange régularisés de manière flexible	17
2.1.1 Objectif de l'article	18
2.1.2 Modèle de mélange de loi	18
2.1.3 L'algorithme EM	19
2.1.4 Modèle FlexMM	19
2.1.5 Application numérique	24
2.2 Segment Anything Model	25
2.2.1 Objectif	25
2.2.2 Base de données	25
2.2.3 Fonctionnement et architecture de SAM	25
2.2.4 Application/Discussion	28
3 Travaux personnels d'amélioration	31

3.1	Initialisation	31
3.2	FlexMM & SAM	33
3.2.1	FlexMM + SAM	33
3.2.2	SAM + FlexMM	34
3.2.3	Image encodée + FlexMM	35
3.3	Nouvelle métrique	36
3.4	Discussion et suggestions	38
3.4.1	Discussion	38
3.4.2	Suggestions	38
Conclusion		40
Glossaire		42
Bibliographie		44
Annexe		45

Table des figures

Figure 1.1 :	Différents types de segmentation d'images	12
Figure 2.1 :	Représentation schématique du graphe de facteurs associé au modèle	21
Figure 2.2 :	Architecture générale du modèle SAM.	26
Figure 2.3 :	Rendus de SAM sur quelques images	29
Figure 3.1 :	K-means	32
Figure 3.2 :	Aléatoire	32
Figure 3.3 :	K-means	32
Figure 3.4 :	DBSCAN	32
Figure 3.5 :	K-means	32
Figure 3.6 :	KNN	32
Figure 3.7 :	K-means	33
Figure 3.8 :	SAM	33
Figure 3.9 :	(a) Image originale	33
Figure 3.10 :	(b) SAM	33
Figure 3.11 :	(c) FlexMM +	33
Figure 3.12 :	Segmentation avec SAM + FlexMM pour différents nombres de classes	34
Figure 3.13 :	Segmentation avec FlexMM seul pour différents nombres de classes	35
Figure 3.14 :	Image originale	35
Figure 3.15 :	Image segmentée	35
Figure 3.16 :	Exemple d'évolution de l'entropie moyenne	36
Figure 3.17 :	Exemple d'évolution de la carte d'entropie	37
Figure 3.18 :	Le modèle SMM(mélange utilisant des distributions de Student-t sans lissage)	54
Figure 3.19 :	Le modèle GMM(mélange utilisant des distributions normales sans lissage)	55
Figure 3.20 :	Résultats obtenus dans l'article	56

Liste des tableaux

Tableau 1.1 :	Matrice de confusion pour un classificateur binaire	15
---------------	---	----

Tableau 2.1 :	Architecture de l'encodeur d'image dans SAM	26
Tableau 2.2 :	Structure de l'encodeur de <i>prompts</i> dans le modèle SAM	27
Tableau 2.3 :	Résumé des étapes de la génération automatique des masques	28
Tableau 2.4 :	Résultats de segmentation avec SAM pour différentes images	29
Tableau 3.1 :	Comparaison des performances de segmentation entre SAM seul et la combinaison FlexMM + SAM	34
Tableau 3.2 :	SAM+FlexMM	35
Tableau 3.3 :	FlexMM	35
Tableau 3.4 :	Résultats aRI pour chaque image, modèle et valeur de K	52
Tableau 3.5 :	Résultats F-score pour chaque image, modèle et valeur de K	53

INTRODUCTION

Le traitement d'images joue aujourd'hui un rôle important dans de nombreux domaines. Selon [1], le processus de traitement d'images est défini comme l'ensemble des méthodes et outils permettant de décrire quantitativement le contenu d'une image numérique. L'une des opérations capitales d'un processus de traitement d'images est la segmentation d'images numériques. Elle se situe avant l'étape d'analyse et d'interprétation. Le traitement d'images est apparu dans les années 1920. De façon simple, la segmentation consiste à diviser une image en régions homogènes ou à isoler des objets d'intérêt. La segmentation d'images, du point de vue de la recherche, a commencé à se développer principalement dans les années 1960 et a connu une évolution importante ces dernières années, notamment avec le développement de l'intelligence artificielle. Au cours de cette évolution, plusieurs méthodes ont été proposées pour effectuer cette tâche, basées sur différentes techniques. Néanmoins, on ne peut pas affirmer, sans aucun doute raisonnable, que ces méthodes de segmentation soient universelles et efficaces à 100 %.

Parmi les méthodes proposées, on trouve celles basées sur la classification non supervisée. Plusieurs techniques permettent de la mettre en œuvre, telles que le clustering, l'algorithme des *K-means*, ainsi que d'autres approches statistiques ou algorithmiques visant à regrouper les pixels en fonction de leur similarité statistique. C'est dans cette optique que McLachlan et al. dans [2] proposent une technique basée sur les modèles de mélanges finis de lois qui constituent une approche souple pour améliorer cette approche vue qu'elle ne tient pas compte de la topologie des données. Jonathan Vacher et al dans [3] ont proposé une nouvelle formulation des mélanges finis probabilistes qui permet d'imposer n'importe quelle topologie sur les probabilités de classe, et qui offre une optimisation computationnellement efficace des paramètres du modèle. Ces méthodes permettent de modéliser de manière rigoureuse la distribution des intensités de pixels, et elles sont capables d'intégrer des connaissances a priori sur les distributions spatiales. Cependant, elles sont limitées dans leur capacité à capturer les relations complexes et les dépendances spatiales entre les objets d'une image, telles que les variations locales des textures ou les interactions entre les objets. De plus, ces méthodes peinent à s'adapter à des images très variées ou dynamiques, comme celles rencontrées dans des environnements réels.

Face à ces défis, les méthodes modernes basées sur des réseaux de neurones profonds, en parti-

culier les architectures spécialisées telles que U-Net, SegNet ou Segment Anything de Meta [4] [5] [6], ont révolutionnées la segmentation d'images. Ces approches permettent d'exploiter les informations contextuelles et structurelles à travers des modèles complexes, capables de capturer les dépendances spatiales et contextuelles présentes dans l'image. Elles offrent ainsi des segmentations plus précises en apprenant automatiquement à partir de grandes quantités de données annotées. Toutefois, cette stratégie présente également des limites. Les réseaux de neurones nécessitent de vastes ensembles de données annotées pour l'entraînement, dont l'obtention peut être coûteuse et chronophage. De plus, la nature des décisions prises par ces modèles manque souvent de transparence, rendant l'interprétation des résultats difficile pour les utilisateurs humains.

Ce travail s'inscrit dans cette dynamique et a pour objectif de proposer une approche hybride, combinant les atouts des modèles probabilistes classiques avec la flexibilité et la puissance des réseaux de neurones profonds. L'intégration de techniques de modélisation probabiliste dans un cadre neuronal permet en effet de mieux quantifier l'incertitude des segmentations et d'obtenir des résultats plus fiables dans des situations complexes.

Trois chapitres composent ce mémoire. Dans le premier chapitre, une présentation et un état de l'art de la segmentation d'images sont proposés, ainsi que les métriques à utiliser pour évaluer les résultats. Dans le deuxième chapitre, nous présentons les deux principaux articles sur lesquels ce travail est basé. Il s'agira d'abord d'une reprise des démonstrations théoriques exposées dans l'article de référence, suivie d'une présentation de l'implémentation réalisée. Enfin, les résultats obtenus seront analysés afin de valider leur conformité avec ceux de l'article. Le troisième chapitre est consacré à nos travaux personnels d'amélioration. Nous exposons d'abord les motivations qui ont guidé nos choix de modification. Ensuite, nous présentons les différentes options d'amélioration explorées, ainsi que les pistes de création de nouvelles métriques d'évaluation. Enfin, nous détaillons les résultats obtenus après cette intégration et procédons à une comparaison avec ceux de l'article d'origine, dans le but de mettre en évidence les apports de notre contribution. Pour terminer, une conclusion générale et les références bibliographiques sont fournies.

CHAPITRE 1

CADRE GÉNÉRAL

Ce chapitre est consacré à la présentation du cadre général du travail. Il s'agit, dans un premier temps, de poser le contexte et d'exposer la problématique. Ensuite, un état de l'art des méthodes de segmentation existantes sera présenté tout en mettant en évidence leurs principes et leurs limites. Enfin, les principales métriques d'évaluation utilisées pour apprécier la qualité des résultats seront définies.

1.1 Contexte

La segmentation d'images a des applications cruciales dans de nombreux domaines, en particulier ceux où il est nécessaire de détecter, isoler et analyser des objets d'intérêt à partir d'images numériques. En médecine, par exemple, la segmentation est essentielle pour identifier et délimiter des structures anatomiques dans des images médicales, telles que les IRM, les scanners ou les radiographies. Ces techniques facilitent grandement les diagnostics en permettant aux médecins d'examiner des régions spécifiques de manière plus précise. Un exemple typique est la segmentation automatique des tumeurs cérébrales dans les images IRM, qui aide les radiologues à mesurer rapidement leur taille, leur forme et à observer leur évolution au fil du temps. Cela permet d'adapter plus efficacement le traitement des patients en fonction de l'évolution de la tumeur. Dans le domaine de la vision par ordinateur, la segmentation est utilisée pour la reconnaissance d'objets et d'environnements. Elle est particulièrement utile dans des domaines comme la conduite autonome, où des véhicules doivent analyser en temps réel les images provenant de leurs caméras pour identifier des objets tels que les routes, les piétons, les autres véhicules ou des obstacles. La segmentation permet de déterminer quelles régions de l'image correspondent à un objet ou à une zone d'intérêt, ce qui est essentiel pour une conduite autonome sécurisée. La surveillance et la sécurité sont également des domaines où la segmentation d'images joue un rôle essentiel. Elle est utilisée pour détecter des intrusions ou des comportements suspects dans des flux vidéo. Par exemple, un système de surveillance peut segmenter une scène pour distinguer les objets en mouvement, ce qui permet d'identifier une personne qui se déplace dans une zone interdite ou un événement anormal qui nécessite une attention immédiate. La segmentation est également utilisée en

télédétection et en imagerie satellite pour analyser des images des terrains ou des paysages. Elle permet d'étudier l'évolution des paysages, d'identifier des zones touchées par des catastrophes naturelles, ou encore de cartographier des régions pour des études environnementales. Par exemple, après une catastrophe naturelle comme un tremblement de terre ou une inondation, la segmentation peut aider à détecter les zones les plus affectées en analysant les images satellite avant et après l'événement. Elle est aussi utilisée pour la surveillance des cultures agricoles, en détectant les différentes espèces végétales ou en estimant les rendements agricoles à partir d'images satellitaires.

Avec les récentes avancées en intelligence artificielle et en apprentissage profond, la segmentation d'images est devenue plus précise, plus rapide et plus fiable. Il existe aujourd'hui plusieurs méthodes différentes pour effectuer cette tâche qu'elles soient une méthode probabiliste, statistique ou basée sur l'intelligence artificielle qui ont chacune leurs qualités. Dans ce contexte, une question principale se pose : **Comment combiner des modèles probabilistes et des techniques modernes de segmentation pour proposer une méthode perceptuelle efficace, capable de produire de bons résultats sur des images naturelles et de rivaliser avec les approches existantes ?**

Des éléments de réponse à cette problématique seront explorés tout au long de ce mémoire. Avant d'aller plus loin, il est important de se demander : **qu'est-ce que la segmentation d'images exactement ? Et quels est sont ses objectifs ?**

Pour obtenir des informations plus détaillées sur chaque notion abordée dans cette section, il y a les références suivantes qui ont servi à la rédaction de cette partie [7] et [8].

1.2 Segmentation d'images

Il est primordial de comprendre ce qu'est la segmentation d'images et quels sont ses objectifs fondamentaux avant de se lancer dans les méthodes et les techniques de segmentation d'images.

1.2.1 Définition de la segmentation d'images

Selon [7], la **segmentation d'images** consiste à diviser une image en différentes régions ou segments homogènes selon des critères spécifiques afin de simplifier son analyse. Ce procédé permet de localiser des objets et des frontières dans des images, en facilitant ainsi diverses applications allant de la reconnaissance d'objets à l'amélioration de la qualité d'image.

Mathématiquement, selon [9], si R désigne une image. On peut considérer la segmentation d'images comme un processus qui divise R en plusieurs parties $R_1, R_2, \dots, R_n$ de l'image telles que :

- (a) $\bigcup_{i=1}^n R_i = R$
- (b) R_i est un ensemble connexe, pour tout $i = 1, 2, \dots, n$.
- (c) $R_i \cap R_j = \emptyset$ pour tout $i \neq j$.
- (d) $Q(R_i) = \text{VRAI}$ pour tout i .
- (e) $Q(R_i \cup R_j) = \text{FAUX}$ pour toutes régions adjacentes R_i et R_j .

Où $Q(R_k)$ est un prédicat logique défini sur les points de l'ensemble R_k , et $\emptyset$ est l'ensemble vide. Autrement dit, Q exprime une condition ou une propriété que doivent satisfaire les pixels de R_k (par exemple : intensité au-dessus d'un certain seuil, appartenance à une zone géographique précise, cohérence spatiale, etc.). Le résultat de $Q(R_k)$ est donc vrai pour les pixels qui vérifient cette propriété et faux pour les autres. Si $Q(R_k) = \emptyset$, cela signifie qu'aucun pixel de la région R_k ne satisfait la condition imposée par Q .

La segmentation est dite **sémantique** si elle consiste à attribuer une classe sémantique à chaque pixel, sans générer d'autres éléments contextuels ou d'informations liées aux objets. Sinon, elle est dite **d'instance**, c'est-à-dire l'algorithme délimite la forme exacte de chaque instance d'objet distincte.

La segmentation est dite **perceptuelle** lorsque l'algorithme essaie d'imiter la manière dont un être humain percevrait et découperait visuellement une image. L'objectif est que la machine partitionne l'image en zones visuellement cohérentes comme le ferait l'œil humain selon des critères perceptifs tels que la continuité, la proximité ou la similarité.

L'appartenance à un certain type de segmentation ne limite pas l'application complémentaire d'une autre approche. Uniquement la segmentation perceptuelle a été étudiée durant ce travail.

La figure suivante illustre la segmentation sémantique et segmentation d'instance.



Fig. 1.1 : Différents types de segmentation d'images

Source : <https://www.ibm.com/fr-fr/think/topics/image-segmentation>

Sur la figure ci-dessus, l'image (a) correspond à l'image d'origine, (b) montre une segmentation sémantique où chaque pixel est associé à une classe d'objet, et (c) illustre une segmentation d'instance distinguant chaque objet individuellement même s'ils appartiennent à la même classe. Une classe désigne une couleur.

1.2.2 Objectifs de la segmentation d'images

Il faut noter que la segmentation parfaite n'existe pas. Il s'agit bien évidemment que la région se rapproche de la notion d'objet, au sens courant du terme. Nonobstant, les attributs acceptables qu'on cherche à obtenir dans un algorithme de segmentation sont :

- **Stabilité** : La segmentation obtenue doit rester stable malgré de légères variations des conditions d'acquisition, telles que le bruit, l'éclairage ou l'angle de prise de vue
- **Régularité** : Les régions segmentées doivent être faciles à manipuler, avec une taille suffisante et une forme relativement régulière.

Les principaux objectifs de la segmentation d'images sont entre autres :

- Fournir des régions homogènes selon un critère spécifique (intensité, texture, couleur, etc.);
- Extraire et/ou séparer les différentes entités ou objets présents dans l'image;
- Localiser les contours des régions ou objets d'intérêt;
- Préparer l'image à des traitements spécifiques (analyse, compression, reconnaissance, etc.);
- Faciliter l'interprétation du contenu visuel de l'image;
- Réduire le bruit pour améliorer la qualité des données segmentées.

1.3 État d'art de la segmentation

Comme décrit brièvement précédemment, il existe plusieurs principales familles d'algorithmes de segmentation, chacune adaptée à des types d'images et des contraintes particulières.

1.3.1 Segmentation de région non texturées (SRNT)

Les méthodes de segmentation basée sur les régions consistent à chercher l'ensemble des pixels qui partagent des propriétés analogues. La segmentation correspond donc aux parties connexes de ces ensembles. Deux techniques de cette approche peuvent être distinguées :

Segmentation par croissance de région (*Region Growing*) : Cette méthode a pour but de sélectionner des pixels de l'image, appelé noyau, autour duquel on fait croître la région en ajoutant les pixels voisins dont la différence d'intensité reste inférieure à un seuil donné. Les noyaux sont généralement choisis de manière optimale selon des critères qui peuvent nécessiter des résultats de segmentation antérieurs. Ces méthodes souffrent de choix du critère d'arrêt de la croissance. Elles n'aboutissent pas à des partitions car la propriété $R = \bigcup_{i=1}^n R_i$ n'est généralement pas satisfaite. Les méthodes de division et fusion permettent de pallier ces insuffisances.

Décomposition et fusion (*Split & Merge*) : La méthode consiste d'abord de diviser l'image récursivement en sous-régions jusqu'à homogénéité, puis des régions voisines sont fusionnées si leur variance σ_R^2 est inférieure à un seuil. Mais des détails fins peuvent être ignorés ou mal segmentés si la grille selon laquelle la division a été faite ne les capture pas bien.

1.3.2 Segmentation basée sur les contours

La segmentation par contours a pour objectif de détecter les discontinuités d'intensité pour en extraire les bords des objets. Elle analyse l'image et identifie les endroits où la luminosité ou la couleur change brusquement. Les filtres de détection de contour peuvent être utilisés pour identifier les différentes régions de l'image. La segmentation par contours est efficace pour isoler des objets aux limites bien définies et pour suivre des objets mobiles dans une séquence vidéo. En revanche, elle peut être sensible au bruit qui crée des faux bords et moins efficace si les frontières des objets sont floues ou incomplètes. Comme exemple, il y a le modèle *Snake*[10] qui consiste à représenter un contour sous

forme de courbe paramétrique $\mathbf{v}(s) = (x(s), y(s))$, puis à définir une fonction d'énergie :

$$E_{\text{snake}} = E_{\text{interne}} + E_{\text{externe}}$$

- **Énergie interne** : contrôle la régularité et la continuité de la courbe (évite qu'elle ne devienne trop irrégulière).
- **Énergie externe** : attire la courbe vers les contours détectés dans l'image (par exemple, vers les gradients élevés).

Le principe est de minimiser E_{snake} afin que la courbe épouse les contours des objets dans l'image.

1.3.3 Segmentation par classification ou seuillage

Elle consiste à regrouper les pixels de niveaux semblables, indépendamment des relations de connectivité qui peuvent les lier. La technique de seuillage d'histogramme, qui constitue la majorité des méthodes de segmentation par classification, s'appuie sur l'hypothèse que les régions de niveaux de gris uniforme produisent des modes suffisamment significatifs de l'image pour que l'on puisse les caractériser directement par les valeurs limites des pixels qui les composent. Il suffit alors de délimiter l'image entre ces deux bornes pour en extraire les régions. La segmentation par classification regroupe des méthodes telles que le seuillage, la méthode d'Otsu, le clustering comme les K-mean, ainsi que des approches supervisées utilisant des classificateurs ou des réseaux de neurones simples. Certains de ces méthodes ont pour inconvénient de ne pas tenir compte de la topologie de l'image¹ autrement dit, elles fournissent des segmentations déformées ou fragmentent les objets de manière incohérente.

1.3.4 Segmentation basée sur l'apprentissage automatique

Les méthodes modernes exploitent l'apprentissage automatique, notamment le deep learning, pour réaliser la segmentation. On entraîne des modèles sur de nombreuses images annotées, c'est-à-dire que chaque pixel a une étiquette, afin qu'ils apprennent à prédire la classe de chaque pixel. Les réseaux de neurones convolutionnels sont très utilisés. Par exemple, l'architecture U-Net combine un encodeur, qui extrait les détails locaux, et un décodeur, qui reconstitue la segmentation globale, ce qui permet de gérer des formes d'objets complexes. D'autres architectures avancées sont également utilisées, comme le Segment Anything Model (SAM) de Meta AI, qui a fait l'objet d'étude dans ce travail.

La question suivante se pose naturellement. *Quelles méthodes choisir donc pour avoir une meilleure segmentation ?*

Le choix d'une méthode de segmentation dépend de plusieurs éléments, comme le type d'image (qualité des contours, texture, éclairage), les conditions dans lesquelles elle a été prise en présence de bruit ou pas, ce que l'on souhaite faire après la segmentation, reconnaissance d'objets ou compression, les informations à extraire, ainsi que les contraintes techniques comme temps de calcul, mémoire disponible, etc.

¹ La topologie de l'image désigne la structure de connectivité et les relations spatiales entre les pixels ou les régions, indépendamment de leurs valeurs d'intensité ou de couleur.

1.4 Métriques pour l'évaluation

L'évaluation des performances d'un algorithme de segmentation repose sur des métriques quantitatives permettant de comparer les résultats obtenus avec une vérité terrain si elle est disponible. Parmi les méthodes les plus couramment utilisées, on peut citer la précision, le rappel, le F-score et l'indice de Rand ajusté (ARI). Ces indicateurs mesurent différentes dimensions de la qualité de la segmentation.

1.4.1 La matrice de confusion

La matrice de confusion est un outil d'évaluation qui permet de comparer les classes prédites par un modèle avec les classes réelles des données. Elle regroupe les prédictions correctes et incorrectes en différentes catégories : les vraies prédictions positives, les vraies négatives, ainsi que les erreurs de classification appelées faux positifs et faux négatifs.

	Prédiction : Positif	Prédiction : Négatif
Vérité : Positif (P)	Vrai Positif (TP)	Faux Négatif (FN)
Vérité : Négatif (N)	Faux Positif (FP)	Vrai Négatif (TN)

Tab. 1.1 : Matrice de confusion pour un classificateur binaire

Dans le cas d'une segmentation binaire, la matrice de confusion montre combien de pixels ont été correctement ou incorrectement classés entre les deux classes (par exemple, objet vs fond), permettant d'évaluer les erreurs comme les faux positifs et faux négatifs. Pour la segmentation multi-classes, on construit une matrice $K \times K$ où chaque élément $M_{i,j}$ correspond au nombre de pixels appartenant à la classe i (vraie classe) mais prédits comme classe j , ce qui permet d'identifier les confusions entre plusieurs classes et de calculer des métriques par classe ou globales.

$$\mathbf{M} = \begin{bmatrix} M_{1,1} & M_{1,2} & \cdots & M_{1,K} \\ M_{2,1} & M_{2,2} & \cdots & M_{2,K} \\ \vdots & \vdots & \ddots & \vdots \\ M_{K,1} & M_{K,2} & \cdots & M_{K,K} \end{bmatrix}$$

1.4.2 Précision

La précision mesure la proportion de pixels correctement identifiés comme appartenant à une classe donnée parmi l'ensemble des pixels prédits comme appartenant à cette classe. Une haute précision indique un faible taux de faux positifs.

$$\text{Précision} = \frac{TP}{TP + FP}.$$

1.4.3 Rappel

Le rappel ou la sensibilité est la proportion de pixels réellement appartenant à la classe cible qui ont été correctement prédits par l'algorithme. Un rappel élevé signifie que peu de pixels ont été mal classés,

autrement dit, un faible taux de faux négatifs.

$$Rappel = \frac{TP}{TP + FN}.$$

1.4.4 F-Score F

Le F-score ou f-mesure est la moyenne harmonique pondérée de la précision et du rappel qui donne deux fois plus de poids à la précision qu'au rappel.

$$F_1 = \frac{2 \times Precision \times Rappel}{Precision + Rappel}$$

Cette quantité atteint 1 si précision et rappel sont parfaits, et tend vers 0 si l'une des deux mesures est nulle. Cette métrique est particulièrement adaptée lorsque l'on souhaite privilégier la fiabilité des segments positifs (précision) au détriment d'un léger manque de couverture (rappel).

1.4.5 Indice de Rand ajusté (ARI)

L'indice de Rand ajusté (Adjusted Rand Index) permet d'évaluer la similarité entre la vérité terrain et la segmentation de l'algorithme, en tenant compte du hasard. Il prend une valeur entre 0 et 1. La valeur 1 indique une correspondance parfaite entre les deux segmentations.

$$aRI = \frac{TP + TN}{TP + FP + FN + TN}$$

Ce chapitre a permis de comprendre ce qu'est la segmentation, son importance, ses objectifs et de montrer qu'il existait de nombreuses approches pour segmenter les images, et que chacune de ces méthodes repose sur des principes différents et s'applique selon le type d'image et les besoins du traitement qui ont leur avantage et inconvénients. Le choix de méthode à utiliser, n'est pas une tâche aisée, car il dépend de plein paramètres. On a également abordé les métriques d'évaluation couramment utilisées pour mesurer la qualité d'une segmentation, telles que la matrice de confusion, la précision, le rappel, le F-score, et l'indice de Rand ajusté.

CHAPITRE 2

PRÉSENTATION DES ARTICLES DE RÉFÉRENCE

Pour ce mémoire, deux articles ont été retenus comme références principales orientant notre démarche méthodologique. Ces travaux font avancer la recherche en matière de segmentation d'images par voie d'utilisation de modèles probabilistes d'une part et d'autres procédés d'apprentissage profond d'autre part.

Le premier article proposé par Jonathan Vacher et al.[3] traite des modèles de mélange de lois pour la segmentation non supervisée en tenant compte de la topologie et des variations d'échelle dans les images naturelles. Le second article est intitulé Segment Anything Model (SAM)[4], un modèle récent développé par Meta AI pour la segmentation universelle d'images. Ce modèle se distingue par sa capacité à segmenter efficacement une grande variété d'images et d'objets sans pour autant besoin d'un entraînement spécifique à chaque type de données ce qui lui donne la qualification d'un outil très polyvalent et performant dans le domaine de la segmentation d'images.

Dans ce chapitre, nous présentons successivement le contexte, les objectifs, les méthodologies et brièvement les résultats des deux articles qui constituent la base théorique des développements et expérimentations suivants.

2.1 Modèles de mélange régularisés de manière flexible

Dans la pratique, les images que nous souhaitons segmenter contiennent souvent du bruit, des textures complexes ou des objets qui se chevauchent. Les méthodes classiques de segmentation non supervisée, comme les mélanges gaussiens, clustering et autres, traitent chaque pixel de manière indépendante et produisent parfois des résultats dispersés ou trop hétérogènes. Pour obtenir des régions plus cohérentes et régulières, il est donc nécessaire d'introduire une forme de lissage spatial de manière à ce que chaque pixel tienne compte de ce que font ses voisins. C'est dans ce but que Vacher et ses collègues ont proposé un modèle de mélanges probabilistes enrichi d'une régularisation, permettant de mieux refléter la continuité des objets dans l'image.

2.1.1 Objectif de l'article

L'objectif principal de l'article est de proposer une formulation générale des modèles de mélanges probabilistes, capable d'imposer des contraintes topologiques sur les probabilités de classe, d'intégrer la notion de multi-échelle et d'optimiser efficacement les paramètres du modèle tout en respectant la structure des données.

Notations

Dans cette section les notations suivantes sont utilisées :

- H , N , K , et D désignent respectivement le nombre de couches, le nombre d'échantillons, le nombre de classes, et la dimension de l'espace de représentation des données.
- Une variable aléatoire est notée par une lettre majuscule, par exemple X , tandis que sa fonction de densité de probabilité est notée P_X .
- Un échantillon particulier de la variable X est noté x_n , pour $1 \leq n \leq N$.
- Les lettres en gras désignent des vecteurs ou collections de variables, par exemple $\mathbf{b} = (b_1, \dots, b_K)$.
- Le simplexe K -dimensionnel, noté Δ_K , représente l'ensemble des vecteurs de probabilités de dimension K , c'est-à-dire :

$$\Delta_K = \left\{ \mathbf{p} \in \mathbb{R}^K \left| \sum_{k=1}^K p_k = 1, p_k \geq 0 \forall k \right. \right\}.$$

- Un ensemble de N échantillons est noté $(x_n)_{1 \leq n \leq N}$, ou plus simplement $(x_n)_n$, voire $\mathbf{x}$.

2.1.2 Modèle de mélange de loi

Commençons par présenter le modèle de mélange classique développé proposé dans [2].

Définition 2.1.1. On considère un ensemble de données constitué de N échantillons $\{x_n\}_{n=1}^N$, où chaque $x_n \in \mathbb{R}^D$ est un vecteur de caractéristiques de dimension D . Chaque observation x_n est supposée être une réalisation d'une variable aléatoire X_n , dont la distribution suit un modèle de mélange à K composantes. Autrement dit, la densité de probabilité de X_n est une combinaison pondérée de K distributions :

$$P_X(x) = \sum_{k=1}^K p_k P_X^{(k)}(x; \alpha_k)$$

où :

- $P_X^{(k)}(x; \alpha_k)$ désigne la densité de la k -ième composante du mélange, paramétrée par α_k ,
- $p_k \in [0, 1]$ est la probabilité de mélange associée à la classe k , avec $\sum_{k=1}^K p_k = 1$,
- le vecteur $\mathbf{p} = (p_1, \dots, p_K) \in \Delta_K$ appartient au simplexe K -dimensionnel,
- $\mathbf{a} = (\alpha_1, \dots, \alpha_K)$ est l'ensemble des paramètres de toutes les composantes.

À chaque variable X_n est associée une variable latente discrète $C_n \in \{1, \dots, K\}$, représentant la classe à laquelle appartient l'échantillon. Le modèle standard proposé par McLachlan suppose que les C_n

sont indépendantes et identiquement distribuées (i.i.d), et issues d'une loi multinomiale de paramètre $\mathbf{p}$, soit :

$$\mathbb{P}(C_n = k) = p_k \quad \text{pour tout } k \in \{1, \dots, K\}, \quad \text{et tout } n \in \{1, \dots, N\}$$

Ainsi, la loi conjointe de x donnée les paramètres du mélange s'écrit :

$$P_{X|P,A}(x|\mathbf{p}, \mathbf{a}) = \sum_{n=1}^N p_k P_{X^{(k)}}(x; a_k)$$

Le paramètre de modèle est donc $\theta = (\mathbf{p}, \mathbf{a})$, il est classiquement estimé par maximisation de la vraisemblance via l'algorithme EM (Expectation-Maximization).

Pour un échantillon observé x_n variable latente C_k la log-vraisemblance est donnée par :

$$\ell(p, a; (x_n)_n) = \ln \left(\prod_{n=1}^N P_{X,C|P,A}(x_n, C_n | p, a) \right)$$

Les détails relatifs à ce calcul sont présentés en annexe.

NB : La loi utilisée pour modéliser les mélanges dans ce travail est la loi de Student en plus de la loi normale. En effet, plusieurs études [11] [12] ont démontré que cette distribution est plus robuste aux valeurs aberrantes que la loi normale et reproduit de manière plus réaliste certains mécanismes de traitement de l'information similaires à ceux du cerveau humain.

2.1.3 L'algorithme EM

L'algorithme EM (pour Expectation-Maximization) est une méthode itérative d'optimisation utilisée pour estimer les paramètres θ d'un modèle probabiliste lorsqu'il existe des variables latentes C_k non observées, en maximisant la vraisemblance marginale des observations X . Chaque itération se passe en deux étapes. En supposant qu'à l'itération t les paramètres du modèle estimés sont $\theta^{(t)} = (p^{(t)}, a^{(t)})$ on aura donc à l'itération $t + 1$:

- **Étape E(Expectation)** : On calcule l'espérance de la log-vraisemblance complète, conditionnellement aux données observées et aux paramètres estimés à l'itération précédente :

$$Q(\theta, \theta^{(t)}) = \mathbb{E}_{C_n|(X_n)_n, \theta} \left[\ell(p, a; (x_n, C_n)_n) \mid (x_n)_n, \theta^{(t)} \right]$$

- **Étape M(Maximization)** : On met à jour les paramètres en maximisant l'espérance calculée précédemment.

$$\theta^{(t+1)} = \arg \max_{\theta} Q(\theta, \theta^{(t)})$$

2.1.4 Modèle FlexMM

Le modèle proposé par les auteurs repose sur deux approches antérieures présentées dans [13] et [14], dont les détails théoriques sont fournis en annexe. Ces approches constituent la base du modèle

introduit dans cet article tout en étant complétées et généralisées dans une formulation plus souple. Alors quel est, concrètement, le modèle que les auteurs ont proposé ?

Contrairement aux travaux précédents effectués dans [13] et [14], qui proposaient des modèles dépendants spatialement mais difficiles à optimiser, les auteurs introduisent une régularisation des probabilités de mélange par une distribution de Dirichlet (Voir annexe) :

$$\mathbf{p}_n \sim \mathcal{D}(\mathbf{B}_n), \quad \text{où} \quad \mathbf{B}_n = (B_{n,1}, \dots, B_{n,K})$$

Les paramètres $B_{n,k}$ sont définis linéairement à partir des classes latentes C_n :

$$B_{n,k} = u_{n,k} (\mathbf{1}_k(\mathbf{C})) - \mathbf{1}_k(C_n) + 1$$

où :

- $\mathbf{1}_k(C_n)$ est l'indicatrice de la classe k pour C_n ,
- $u_{n,k} : \mathbb{R}^N \rightarrow \mathbb{R}_+$ est une fonction linéaire, choisie par l'utilisateur selon la topologie (voisinage spatial, sémantique, etc.).

Le modèle se base désormais sur trois types de variables principales :

- $\mathbf{X} = (x_n)_{n=1}^N$: les données observées (par exemple, les pixels d'une image ou les vecteurs de caractéristiques),
- $\mathbf{C} = (C_n)_{n=1}^N$: les variables latentes discrètes représentant l'appartenance de chaque observation à une classe,
- $\mathbf{B}$: un champ latent de régularisation spatiale qui encode des interactions entre les observations (voisinage, structure de graphe, lissage spatial, etc.).

L'introduction de $\mathbf{B}$ permet de modéliser explicitement des dépendances locales entre les variables C_n en imposant une certaine cohérence spatiale ou structurelle dans les assignations de classes. Ainsi, la régularisation introduite via $\mathbf{B}$ favorise des partitions plus homogènes et réalistes. Cependant, dans le cadre d'une régularisation des probabilités de mélange p_n sur-paramétrées, la structure graphique probabiliste représentant le modèle dirigé peut contenir des boucles qui posent plusieurs problèmes, notamment une possible incohérence dans la définition du modèle, l'impossibilité de définir une distribution conjointe unique sur l'ensemble des variables. Pour pallier ces difficultés, on propose d'identifier le graphe dirigé initial à un graphe de facteurs. Cette représentation plus générale permet de modéliser explicitement les dépendances et interactions entre variables tout en gérant de façon efficace la présence de boucles.

Deux différents modèles ont été proposé dans ce contexte :

Modèle de mélange simple avec la régularisation

Ici, le modèle n'est rien d'autre de modèle simple présenté précédemment auquel est ajouté la régularisation. D'après le théorème de Hammersley-Clifford [15], si le graphe associé au modèle est un graphe de Markov, alors la distribution conjointe des variables cachées $\mathbf{C}$ et $\mathbf{B}$, des observations $\mathbf{X}$, et des paramètres $\mathbf{p}$ et $\mathbf{a}$, peut s'écrire comme un produit de fonctions potentielles définies sur les cliques du graphe. Ainsi, la distribution conjointe prend la forme suivante :

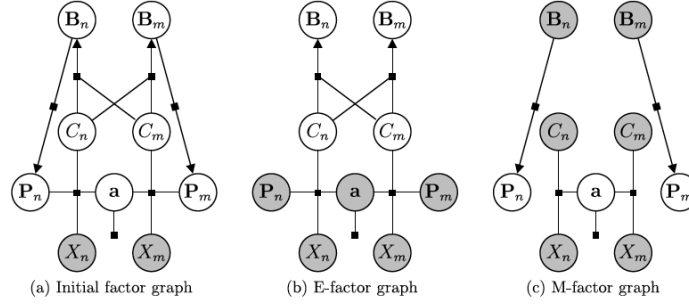


Fig. 2.1 : Représentation schématique du graphe de facteurs associé au modèle
Source : [3]

$$P_{X,C,B,P,A}((x_n)_n, (C_n)_n, (B_n)_n, (\mathbf{p}_n)_n, \mathbf{a}) = \frac{1}{Z} f_A(\mathbf{a}) \prod_{n=1}^N f_{X,C,A,P}(x_n, C_n, \mathbf{a}, \mathbf{p}_n)$$

$$f_{P,B}(\mathbf{p}_n, B_n) f_{B,(C_n)_n}(B_n, (C_n)_n)$$

Avec :

$$f_{X,C,A,P}(x_n, C_n, \mathbf{a}, \mathbf{p}_n) = P_{X,C|A,P}(x_n, C_n | \mathbf{a}, \mathbf{p}_n)$$

$$f_{P,B}(\mathbf{p}_n, B_n) = P_{P|B}(\mathbf{p}_n | B_n)$$

$$f_{B,(C_n)_n}(B_n, (C_n)_n) = P_{B|(C_n)_n}(B_n | (C_n)_n)$$

$$f_A(\mathbf{a}) = P_A(\mathbf{a})$$

Donc :

$$\ell(\mathbf{p}, \mathbf{a}; (x_n, C_n, B_n)_n) = \ln(P_{P,A|X,C,B}(\mathbf{p}, \mathbf{a} | (x_n)_n, (C_n)_n, (B_n)_n))$$

$$= \ln(P_{X,C,B,P,A}((x_n)_n, (C_n)_n, (B_n)_n, (\mathbf{p}_n)_n, \mathbf{a})) - \ln(P_{X,C,B}((x_n)_n, (C_n)_n, (B_n)_n))$$

$$= \sum_{n=1}^N \ln(P_{X,C|P,A}(x_n, C_n | \mathbf{p}_n, \mathbf{a})) + \ln(P_{P|B}(\mathbf{p}_n | B_n)) + \ln(P_{B|(C_n)_n}(B_n | (C_n)_n))$$

$$+ \ln(P_A(\mathbf{a})) - \ln(P_{X,C,B}((x_n)_n, (C_n)_n, (B_n)_n)) - \ln(Z)$$

$$P_{X,C|P,A}(x_n, C_n | \mathbf{p}_n, \mathbf{a}) = \prod_{k=1}^K (p_{n,k} P_{X(k)}(x_n; a_k))^{1_{k(C_n)}}$$

$$P_{B_n|(C_n)_n}(B_n | (C_n)_n) = \delta_{v_n(1_1(C_n), \dots, 1_K(C_n))}(B_n)$$

$$W((x_n, C_n, B_n)_n)$$

On obtient finalement :

$$\begin{aligned} \ell(\mathbf{p}, \mathbf{a}; (x_n, C_n, B_n)_n) = & \sum_{n=1}^N \sum_{k=1}^K 1_k(C_n) [\ln(\mathbf{p}_{n,k}) + \ln(P_{X(k)}(x_n; a_k))] + \ln(P_A(\mathbf{a})) \\ & + \ln(P_{P|B}(\mathbf{p}_n | v_n(1_1(C), \dots, 1_K(C)))) + W((x_n, C_n, B_n)_n) \end{aligned}$$

En calculant le gradient de la log-vraisemblance complète par rapport aux paramètres, puis en résolvant les équations obtenues, on peut déterminer les règles de mise à jour des paramètres du modèle. Ces mises à jour explicites sont présentées dans le théorème suivant.

Proposition 2.1.1. *Pour tout $(n, k) \in \Omega_{N,K} = \{1, \dots, N\} \times \{1, \dots, K\}$, soit $u_{n,k} : \mathbb{R}^N \rightarrow \mathbb{R}$ une fonction linéaire telle que $u_{n,k}([0, +\infty[^N) \subset \mathbb{R}^+$. Si*

$$v_n(1_1(C), \dots, 1_K(C)) = (u_{n,1}(1_1(C)) - 1_1(C_n) + 1, \dots, u_{n,K}(1_K(C)) - 1_K(C_n) + 1),$$

alors les mises à jour des probabilités de mélange sont données par

$$\forall (n, k) \in \Omega_{N,K}, \quad p_{n,k}^{(t+1)} = \frac{u_{n,k}(\tau_{\cdot,k}^{(t)})}{\sum_{k=1}^K u_{n,k}(\tau_{\cdot,k}^{(t)})}.$$

Avec

$$\tau_{n,k}^{(t)} = P(C_n = k | x_n, \theta^{(t)})$$

La preuve de cette proposition est présentée en annexe. La proposition suivante nous assure la stabilité et la convergence de l'algorithme EM. Et il est important de noter que l'algorithme converge très vite généralement au tour de 15 itérations au plus.

Proposition 2.1.2. *L'algorithme EM appliqué à ce modèle est un algorithme de montée, ce qui signifie qu'à chaque itération $t+1$, le logarithme du postérieur incomplet augmente lorsque les paramètres sont mis à jour :*

$$\ln P_{P,A|X}(p^{(t+1)}, a^{(t+1)} | (x_n)_n) \geq \ln P_{P,A|X}(p^{(t)}, a^{(t)} | (x_n)_n).$$

La preuve de cette proposition s'inspire de la démonstration du comportement monotone de la log-vraisemblance dans l'algorithme EM classique [16].

Combinaison de modèles de mélange

Dans les jeux de données réels, un même échantillon peut être représenté par plusieurs vecteurs de caractéristiques $x_n^{(h)}$ ($h = 1, \dots, H$) qui correspondent par exemple aux activations issues des différentes couches d'un réseau de neurones profond. Ces vecteurs sont souvent de dimensions différentes, avec des structures topologiques ou hiérarchiques naturelles. Une approche naïve consisterait à entraîner indépendamment plusieurs modèles de mélange pour chaque vecteur. Toutefois, cela ignorerait la

structure multi-niveaux des données et poserait un problème de permutation des composantes qui est un problème combinatoire coûteux. Il a été proposé d'entraîner ces modèles de mélange en parallèle, en partageant l'information de classe entre eux. Ce partage est assuré par une régularisation conjointe des probabilités de mélange, de sorte que l'information de classe se propage entre les différentes représentations.

Chaque vecteur $x_n^{(h)}$ est modélisé par un mélange de distributions :

$$P(x_n^{(h)}) = \sum_{k=1}^K p_{n,k}^{(h)} P_{X^{(h)}}^{(k)}(x_n^{(h)}; \alpha_k^{(h)}) \quad (2.1)$$

Pour éviter un apprentissage indépendant des poids $p_{n,k}^{(h)}$, on introduit une variable latente $B_n^{(h)}$ dont la distribution est régularisée à l'aide d'une fonction linéaire $v_n^{(h)}$ des affectations de classes $C_n^{(h)}$ des autres couches :

$$P(\mathbf{B}_n^{(h)} \mid (C_n^{(h)})_{n,h}) = \delta_{v_n^{(h)}(\{1_k(C_n^{(h)})\}_{k,h})}(\mathbf{B}) \quad (2.2)$$

avec $v_n^{(h)} : \mathbb{R}^{N \times H \times K} \rightarrow \mathbb{R}^K$

Cela permet de régulariser les probabilités de mélange conditionnelles $P(p_n^{(h)} \mid \mathbf{B}_n^{(h)})$

L'ensemble des paramètres du modèle est estimé par inférence MAP (Maximum a Posteriori) :

$$(\hat{\mathbf{a}}, \hat{\mathbf{p}}) = \arg \max_{\mathbf{a}, \mathbf{p}} \ell(\mathbf{a}, \mathbf{p}; (x_n^{(h)}, C_n^{(h)}, \mathbf{B}_n^{(h)})_{n,h}) \quad (2.3)$$

Avantages Cette approche permet de préserver la cohérence entre les différentes couches, tirer parti de la structure hiérarchique ou topologique des données, éviter le coût computationnel lié au re-étiquetage des composantes.

Dans ce cas la log-vraisemblance s'écrit :

$$\ell(\mathbf{p}, \mathbf{a}; (x_n^{(h)}, C_n^{(h)}, B_n^{(h)})_{n,h}) = \ln \left(P_{A,P|X,C,B} \left(\mathbf{a}, \mathbf{p} \mid (x_n^{(h)})_{n,h}, (C_n^{(h)})_{n,h}, (B_n^{(h)})_{n,h} \right) \right)$$

En supposant que les paires $(x_n^{(h)}, C_n^{(h)})_h$ sont conditionnellement indépendantes sachant $(p_n^{(h)}, a^{(h)})_h$ et que les $p_n^{(h)}$ sont indépendants sachant $B_n^{(h)}$, on peut factoriser cette log-vraisemblance comme suit :

$$\begin{aligned} \ell(\mathbf{p}, \mathbf{a}; (x_n^{(h)}, C_n^{(h)}, B_n^{(h)})_{n,h}) &= \sum_{n=1}^N \sum_{h=1}^H \sum_{k=1}^K 1_k(C_n^{(h)}) \left[\ln(p_{n,k}^{(h)}) + \ln \left(P_{X^{(h)},k}(x_n^{(h)}; a_k^{(h)}) \right) \right] \\ &\quad + \ln \left(P_{P|B} \left(p_n^{(h)} \mid v_n^{(h)} \left(\{1_k(C^{(h)})\}_{k,h'} \right) \right) \right) \\ &\quad + \sum_{h=1}^H \ln P_A(a^{(h)}) + W \left((x_n^{(h)}, C_n^{(h)}, B_n^{(h)})_{n,h} \right) \end{aligned}$$

où W regroupe les termes qui ne dépendent pas de $\mathbf{p}$ ou $\mathbf{a}$. On obtient donc la proposition suivante :

Proposition 2.1.3. *Mise à jour des probabilités de mélange*

Soit $\Omega_{N,K,H} = \{1, \dots, N\} \times \{1, \dots, K\} \times \{1, \dots, H\}$.

Pour tout $(n, k, h) \in \Omega_{N,K,H}$, soit $u_{n,k}^{(h)} : \mathbb{R}^{N \cdot H} \rightarrow \mathbb{R}_+$ une fonction linéaire telle que :

$$\mathbf{v}_n^{(h)} \left(\{1_k(C^{(h)})\}_{k=1, \dots, K, h=1, \dots, H} \right) = \left(u_{n,1}^{(h)}(1_1(C^{(1)}), \dots, 1_1(C^{(H)})) - 1_1(C^{(h)}) + 1, \right. \\ \left. \dots, u_{n,K}^{(h)}(1_K(C^{(1)}), \dots, 1_K(C^{(H)})) - 1_K(C^{(h)}) + 1 \right)$$

Alors, la mise à jour des probabilités de mélange au temps $t + 1$ s'écrit :

$$p_{n,k}^{(t+1,h)} = \frac{u_{n,k}^{(h)} \left(\tau_{\cdot,k}^{(t,1)}, \dots, \tau_{\cdot,k}^{(t,H)} \right)}{\sum_{k'=1}^K u_{n,k'}^{(h)} \left(\tau_{\cdot,k'}^{(t,1)}, \dots, \tau_{\cdot,k'}^{(t,H)} \right)}$$

où $\tau_{n,k}^{(t,h)} = P_{C_n^{(h)} | X_n^{(h)}, \Theta} (k | x_n^{(h)}, \theta^{(t,h)})$ est la probabilité a posteriori de la composante k pour $x_n^{(h)}$ à l'étape E précédente.

La démonstration de cette proposition a été reproduite au cours du stage et peut être trouvée en annexe.

Pour améliorer les modèles de mélange, les images sont d'abord traitées par un réseau de neurones convolutifs (CNN) afin d'en extraire des caractéristiques pertinentes à différents niveaux de détail. Cette extraction s'appuie sur une approche pyramidale, qui capture des informations à plusieurs échelles. Chaque vecteur $\mathbf{x}_n$ de caractéristiques ainsi obtenus, correspond donc à une représentation extraite localement à un certain endroit et à une certaine échelle de l'image. Ils servent ensuite d'entrée aux modèles de mélange pour réaliser une segmentation plus précise et robuste.

2.1.5 Application numérique

Passons donc à l'application afin de comparer nos résultats avec ceux présentés dans l'article.

Base de données

La base de données utilisée pour tester les algorithmes est le BSD500. Elle contient 500 images naturelles chacune annotée par plusieurs experts pour délimiter les contours perceptuels présents. En moyenne, chaque image a été segmentée par cinq annotateurs différents fournissant ainsi des vérités terrain humaines précises et variées pour évaluer les algorithmes de segmentation.

Résultats

L'application des méthodes a été réalisée en suivant les configurations proposées dans l'article, notamment les variantes suivantes :

- GMM (Gaussian Mixture Model) : modèle de mélange gaussien classique, sans lissage spatial,
- SMM (Student Mixture Model) : modèle de mélange utilisant des distributions de Student-t, plus robuste aux valeurs aberrantes, également sans lissage,

- sGMM (smoothed Gaussian Mixture Model) : GMM avec ajout de lissage spatial sur les probabilités a priori,
- sSMM (smoothed Student Mixture Model) : SMM avec lissage spatial.

Nous avons également testé l'extension basée sur le mélange multiple. Les résultats obtenus reproduisent fidèlement ceux présentés dans l'article original [3], confirmant ainsi la robustesse et la validité de la méthodologie. L'ensemble des résultats, incluant les cartes de segmentation et les courbes d'évaluation pour chaque configuration, est présenté en annexe.

Ce modèle a fait ses preuves et un point positif est que ça permet d'avoir des cartes de probabilité, qui ne sont rien d'autre que les probabilités pour qu'un pixel appartienne à une classe donnée. Ces cartes de probabilité peuvent être utilisées pour quantifier l'incertitude autour d'une segmentation. Donc on a autant de cartes de probabilité que de classes, mais la limite majeure de ce modèle est que le nombre de classes doit être donné par l'utilisateur.

2.2 Segment Anything Model

Cette section est consacrée à la présentation des objectifs, du fonctionnement, de la structure et de l'architecture de SAM, ainsi qu'à son intérêt pour notre problématique.

2.2.1 Objectif

Le modèle Segment Anything (SAM) développé par Meta AI en 2023, vise à créer un système de segmentation d'images capable de détecter et segmenter tout objet visible dans une image sans limite de catégorie ou de contexte. Son ambition est d'établir une segmentation universelle et promptable c'est-à-dire pouvant être guidée par différents types d'informations entrées par l'utilisateur. Cette approche vise à dépasser les modèles traditionnels spécialisés sur un nombre restreint de classes en proposant un modèle polyvalent capable de s'adapter à diverses tâches sans réentraînement spécifique.

2.2.2 Base de données

La base de données utilisée pour entraîner le modèle SAM, appelée SA-1B, est un jeu de données massif contenant environ un milliard de masques de segmentation générés automatiquement. Ces masques sont produits à partir d'images haute résolution variées, couvrant de nombreux types d'objets et de scènes. Pour créer ce jeu de données, une version spéciale du modèle SAM a été entraînée plus longtemps avec des données manuelles et semi-automatiques, en utilisant moins de points par masque et un décodeur plus profond afin d'améliorer la qualité des annotations. Ce large dataset permet au modèle d'apprendre à segmenter efficacement une grande variété d'objets dans des contextes diversifiés.

2.2.3 Fonctionnement et architecture de SAM

Le fonctionnement de ce modèle est fondé sur un processus basé sur un Vision Transformer (ViT) qui extrait des caractéristiques visuelles riches et multi-échelles de l'image. Ces caractéristiques sont

ensuite ajoutées aux *embeddings* conçus grâce aux informations fournies par l'utilisateur (*prompt*) via un module de fusion multimodal qui permet au décodeur de générer des masques binaires précis correspondant aux objets ciblés. La première version de ce modèle a environ 1,17 milliard de paramètres.

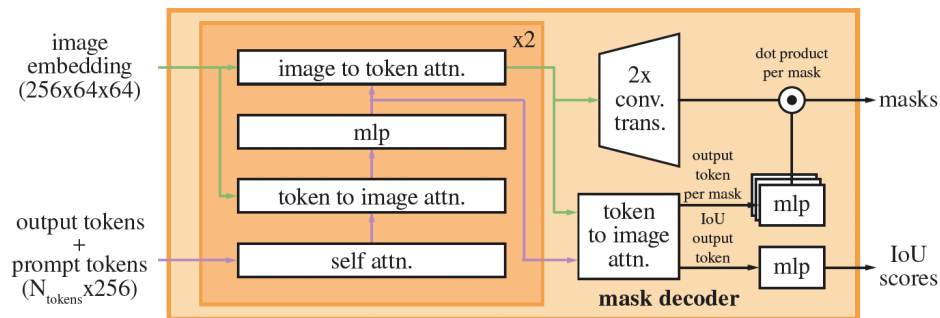


Fig. 2.2 : Architecture générale du modèle SAM.

Source : [4]

SAM est constitué de trois grands modules principaux à savoir :

a) Le Backbone

Le *backbone* est la première étape dont le rôle est de convertir l'image d'entrée en une représentation riche et utile pour guider la segmentation. Pour cela, l'image est tout d'abord découpée en petits carrés de 16×16 pixels qui sont en suite transformés en un vecteur d'*embedding*. Ces vecteurs passent ensuite par plusieurs couches transformeur qui appliquent de l'attention multi-têtes pour capturer les relations à différentes échelles dans l'image. Le résultat est une carte de caractéristiques spatiale (tenseur de forme $(256, 64, 64)$) à haute valeur sémantique qui est transférée aux modules suivants. Le tableau ci-contre présente les différentes sous-étapes du processus ainsi que les éléments constitutifs associés à chacune d'elles.

Étape	Détails
Prétraitement image	- Redimensionnement de l'image à 1024×1024 px. - Ajout de padding si nécessaire pour conserver le ratio d'origine.
ViT-H/16 avec MAE pré-entraîné	- Utilisation de fenêtres locales 14×14 pour l'attention. - 4 blocs d'attention globale répartis uniformément. - Sortie : embeddings spatiaux de taille 64×64.
Réduction des dimensions	- Convolution 1×1 : réduction des canaux à 256. - Convolution 3×3 avec padding : maintien de la résolution spatiale. - Chaque couche est suivie d'une Layer Normalization.
Sortie finale	Tenseur de forme (256, 64, 64) : représentation enrichie pour les modules suivants du modèle.

Tab. 2.1 : Architecture de l'encodeur d'image dans SAM

b) Module encodeur des prompts

Le *Prompt encoder* a pour rôle de convertir les différentes indications fournies par l'utilisateur qu'il s'agisse de points, de boîtes, de masques ou de texte en représentations vectorielles de dimension 256, exploitables par le décodeur de masques. Pour *prompts* discrets comme les points, chaque point est représenté par la somme de son encodage de position et d'un *embedding* appris indiquant s'il appartient au premier plan (*foreground*) ou à l'arrière-plan (*background*). Les boîtes englobantes sont quant à elles codées par une paire de vecteurs, correspondant au coin supérieur gauche et au coin inférieur droit, chacun combinant un encodage de position et un *embedding* spécifique à la position (« *top-left* » ou « *bottom-right* »). Le texte libre est pris en charge via l'encodeur de texte CLIP, permettant ainsi une interaction multimodale entre texte et image. Pour les *prompts* denses, principalement les masques binaires, le processus débute par une réduction de résolution à un quart de celle de l'image originale. Cette représentation est ensuite affinée par deux convolutions successives de taille 2×2 et *stride* 2, réduisant progressivement le nombre de canaux à 4 puis 16. Enfin, une convolution 1×1 projette ce tenseur sur 256 canaux. Chaque couche est suivie d'une activation non linéaire GELU et d'une normalisation de couche (LayerNorm) pour assurer une meilleure stabilité et efficacité de l'apprentissage. Ce traitement permet ainsi d'extraire des *embeddings* riches et structurés des différentes formes de *prompts*, indispensables au fonctionnement précis du modèle SAM.

Type de prompt	Encodage
Point (foreground /background)	- Encodage de position du point + - Embedding appris indiquant le type (avant-plan ou arrière-plan). ⇒ Vecteur final : 256 dimensions.
Boîte englobante (bounding box)	- Coin supérieur gauche : encodage position + embedding "top-left". - Coin inférieur droit : encodage position + embedding "bottom-right".
Texte libre	- Utilisation de l'encodeur de texte CLIP (texte → embedding 256D).
Masque binaire (dense prompt)	- Résolution divisée par 4, puis deux convolutions (2×2 , stride 2) → sorties 4 puis 16 canaux. - Convolution 1×1 pour projection en 256 canaux. - Activations GELU + normalisation après chaque couche.

Tab. 2.2 : Structure de l'encodeur de *prompts* dans le modèle SAM

c) Le Décodeur de Masques

Le décodeur de SAM est un module léger à deux couches qui prend en entrée les *embeddings* de l'image générés par l'encodeur et les *embeddings* des *prompts*. Grâce à des mécanismes d'attention croisée, il met à jour ces *embeddings* pour en extraire l'information pertinente. Ensuite, il utilise les *embeddings* mis à jour pour prédire dynamiquement un ou plusieurs masques, en combinant les indices visuels extraits de l'image avec les indications fournies par l'utilisateur.

Étape	Détails
Découpage (Cropping)	- Grille régulière de 32×32 points sur l'image complète. - 20 découpes zoomées : fenêtres 2×2 et 4×4 avec grilles 16×16 et 8×8 points. - Utilisation des images haute résolution originales. - Suppression des masques touchant les bords internes des découpes.
Suppression non maximale (NMS)	- NMS en deux phases : intra-découpe puis inter-découpe. - Classement selon le score IoU prédit (dans les découpes) et selon le niveau de zoom (entre découpes). - Seuil NMS fixé à 0.7.
Filtrage	- Confiance : masque conservé si score IoU prédit ≥ 0.8. - Stabilité : IoU entre masques seuilés à -1 et +1 ≥ 0.95. - Taille : élimination des masques couvrant $\geq 95\%$ de l'image.
Post-traitement	- Suppression des petits composants connectés < 100 pixels. - Remplissage des trous dans les masques de taille < 100 pixels.
Modèle de génération automatique	- Version spéciale de SAM entraînée plus longtemps (177 656 itérations). - Utilisation uniquement de données manuelles et semi-automatiques. - 4 points échantillonnés par masque (au lieu de 9). - Décodage du masque avec 3 couches au lieu de 2.

Tab. 2.3 : Résumé des étapes de la génération automatique des masques

Notre projet nécessite une segmentation flexible et précise capable de gérer des images avec une grande diversité d'objets et de contextes sans avoir à entraîner un modèle spécifique pour chaque cas, ce qui fait de SAM le candidat idéal.

Contrairement à d'autres méthodes qui nécessitent de spécifier le nombre de classes à l'avance, SAM détermine automatiquement ce nombre en analysant l'image. Cela lui permet de capturer une grande variété de régions, mais conduit parfois à une sur-segmentation, en identifiant un nombre excessif de classes. Un même pixel peut apparaître dans plusieurs masques, ce qui permet à SAM d'identifier un objet situé sur un autre. Les résultats obtenus sont impressionnants par leur précision et leur capacité à isoler finement les objets dans l'image.

2.2.4 Application/Discussion

SAM a été appliqué sur quelques images naturelles notamment ceux utilisées pour tester la méthode précédente. Et voici les résultats trouvés :

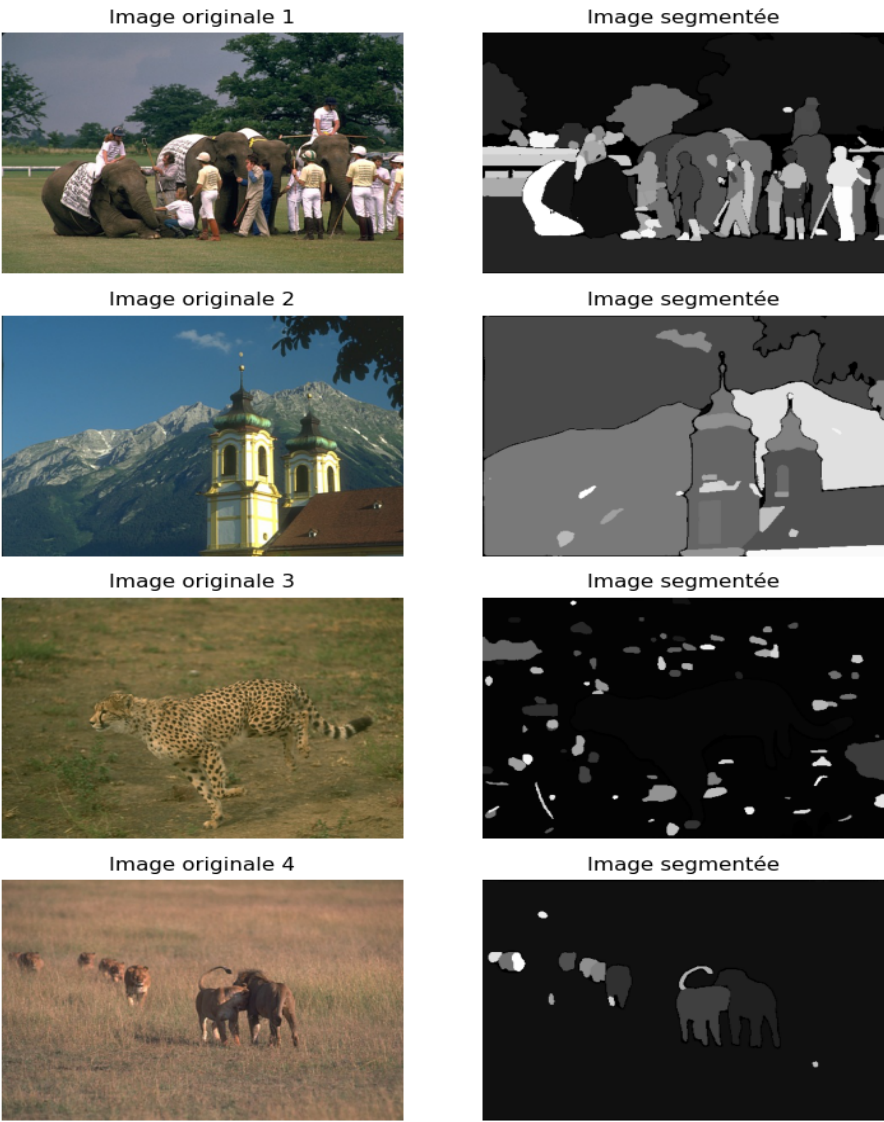


Fig. 2.3 : Rendus de SAM sur quelques images

Les métriques calculées sont consignées dans le tableau suivant :

Image	Nb classes	F-Score	ARI
Image 1	109	0.014	0.80
Image 2	48	0.034	0.62
Image 3	80	0.008	0.45
Image 4	16	0.025	0.32

Tab. 2.4 : Résultats de segmentation avec SAM pour différentes images

Ce tableau montre que SAM peut identifier un grand nombre de classes dans une image (jusqu'à 109 pour l'image 1), ce qui illustre sa capacité à segmenter finement. Cependant, les F-Scores très faibles

(inférieurs à 0.04) indiquent que les segments générés ne correspondent pas toujours bien aux vérités terrain. L'Indice d'Adjusted Rand (ARI), qui évalue la qualité de l'appariement entre la segmentation automatique et les annotations réelles, varie fortement selon les images, allant de 0.32 à 0.80. Cela suggère que, même si SAM est capable de détecter de nombreux objets, il manque parfois de pertinence sur certaines images.

Dans ce chapitre, nous avons présenté deux approches avancées pour la segmentation d'images, l'un modèle basé sur des techniques de mélange et le modèle SAM. Le premier repose sur des principes probabilistes ou hybrides pour modéliser la variabilité des objets. Le second, SAM, adopte une approche plus universelle, en combinant un puissant encodeur visuel, un encodeur de *prompts* multimodal, et un décodeur efficace permettant la segmentation à la demande.

Chacun de ces modèles présente des atouts distincts. Tandis que le modèle de mélange excelle dans des contextes spécialisés et adaptatifs, SAM se distingue par sa capacité à segmenter efficacement n'importe quel objet dans n'importe quelle image, grâce à un apprentissage massif et à une architecture modulaire. Cette double présentation permet de mieux cerner les choix architecturaux possibles en segmentation et oriente les perspectives selon les objectifs visés qui sont généralisation, précision, ou adaptabilité. On peut aussi affirmer que SAM est plus adapté à la détection d'objets que pour la segmentation.

CHAPITRE 3

TRAVAUX PERSONNELS D'AMÉLIORATION

Ce chapitre est consacré à la présentation de l'ensemble des travaux réalisés dans le but d'améliorer les performances du modèle FlexMM. Il aborde dans un premier temps, l'amélioration de l'initialisation de l'algorithme. Ensuite, une combinaison potentielle des deux modèles. Enfin, l'analyse des différentes informations fournies par le modèle FlexMM dans le but de proposer de nouvelles métriques d'évaluation pour mesurer la qualité des résultats obtenus. Tous les travaux seront testés avec les données de la base de données BSD500.

3.1 Initialisation

Une étape importante de tous les algorithmes fonctionnant de façon itérative est l'initialisation des paramètres. Dans le modèle FlexMM, pour l'initialisation, on réalise une Analyse en Composante Principale (ACP) pour réduire la dimension puis, l'algorithme K-means est utilisé pour initialiser les paramètres des modèles de mélange comme les moyennes des clusters. Cela permet de commencer l'apprentissage avec des centres de clusters bien placés facilitant ainsi la convergence du modèle. Dans plusieurs algorithmes informatiques, l'initialisation peut jouer un rôle capital dans la convergence et la précision des résultats. Pour cela, d'autres initialisations ont été testées pour voir leur effet. Entre autres :

Initialisation aléatoire : Dans tous les cas, une initialisation aléatoire a été testée, c'est-à-dire que les classes ont été attribuées au départ de façon aléatoire et uniforme. Chaque pixel avait la même probabilité d'appartenir à une classe donnée. Cette initialisation a eu une incidence sur la segmentation obtenue, montrant que la classe initiale des pixels est vraiment importante pour obtenir une bonne segmentation.



Fig. 3.1 : K-means



Fig. 3.2 : Aléatoire

DBSCAN : Un algorithme de clustering basé sur la densité qui est également un algorithme de classification non supervisée comme celui utilisé par les auteurs. Mais les résultats ont montré que ce changement n'a pas eu d'influence, que ce soit sur la convergence(nombre d'itération), la segmentation, le temps d'inférence.



Fig. 3.3 : K-means



Fig. 3.4 : DBSCAN

Classification supervisée : Plusieurs méthodes différentes de classification supervisée ont été utilisées. On a, par exemple, les k-plus proches voisins, SVM, XGBoost. Avant de les utiliser pour classer, on entraîne ces modèles sur une partie de la base de données en prenant comme label l'une des vérités terrain. Mais on constate également que ce changement n'a pas eu une grande influence sur la convergence ou la segmentation, mais a doublé le temps d'inférence de l'algorithme.



Fig. 3.5 : K-means



Fig. 3.6 : KNN

Initialisation avec SAM : L'idée est d'utiliser les masques de segmentation générés par SAM pour initialiser le modèle. On utilise pas tous les masques mais les k (nombre de classes) premiers ayant le plus de pixels. Il a été constaté qu'il y a pas aussi grande différence entre les résultats, mais la convergence est rapide.



Fig. 3.7 : K-means



Fig. 3.8 : SAM

Il est donc évident que cette méthode n'est pas sensible à l'initialisation, sauf lorsqu'il s'agit d'une initialisation aléatoire. En effet, quelle que soit la méthode d'initialisation, l'algorithme converge en un nombre similaire d'itérations et les résultats restent proches les uns des autres. Cependant, il est important que l'initialisation soit effectuée selon une certaine logique afin d'assurer une segmentation cohérente. Il est donc préférable de privilégier l'initialisation la plus simple à mettre en œuvre (par exemple K-means, DBSCAN) tout en respectant cette logique.

3.2 FlexMM & SAM

L'objectif est d'associer les deux modèles pour obtenir un de plus performant, ce qu'on a d'ailleurs commencé par effectuer dans la section précédente. Hormis le fait d'utiliser les masques de SAM, trois idées ont été creusées.

3.2.1 FlexMM + SAM

Dans un premier temps, l'idée a été d'utiliser la segmentation obtenue avec le modèle de mélange de lois en argument pour le modèle SAM c'est-à-dire, on utilise le modèle FlexMM survit de SAM. Il a été constaté que, suite à cela, on obtient une segmentation avec beaucoup plus de classes qui double voire triple le nombre de classes trouvées si l'image originale est segmentée directement avec SAM. Voici quelques-unes des résultats obtenus :



Fig. 3.9 : (a) Image originale



Fig. 3.10 : (b) SAM

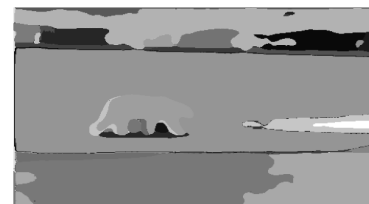


Fig. 3.11 : (c) FlexMM +

Les figures ci-dessus présentent une comparaison visuelle des résultats de segmentation appliqués à une même image. La sous-figure (a) montre l'image originale en entrée. La sous-figure (b) illustre la segmentation obtenue avec le modèle SAM, tandis que la sous-figure (c) correspond à la segmentation réalisée avec la combinaison FlexMM + SAM. Cette comparaison permet de visualiser les différences de précision et de détails entre les deux approches de segmentation.

Le tableau suivant résume les métriques calculées.

Méthode	Nb classes	F-Score	ARI
SAM	19	0.52	0.95
FlexMM + SAM	39	0.139	0.87

Tab. 3.1 : Comparaison des performances de segmentation entre SAM seul et la combinaison FlexMM + SAM

Bien que la combinaison FlexMM + SAM détecte plus de classes, elle présente une baisse notable du F-Score, indiquant une précision de segmentation moins fiable, malgré un ARI encore élevé.

3.2.2 SAM + FlexMM

Ici, on effectue l'opération inverse de celle réalisée précédemment, c'est-à-dire que SAM est appliqué après FlexMM. Il a été remarqué que cette méthode peine à détecter correctement les objets, bien que les résultats métriques semblent indiquer le contraire. Cela pourrait s'expliquer par le fait que SAM génère des masques où un même pixel peut appartenir à plusieurs masques. L'exemple suivant illustre ce que cas de figure.

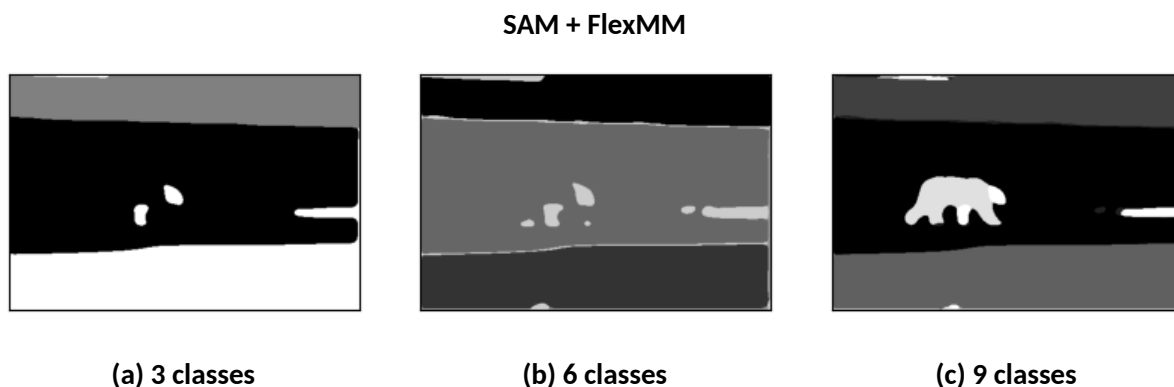


Fig. 3.12 : Segmentation avec SAM + FlexMM pour différents nombres de classes

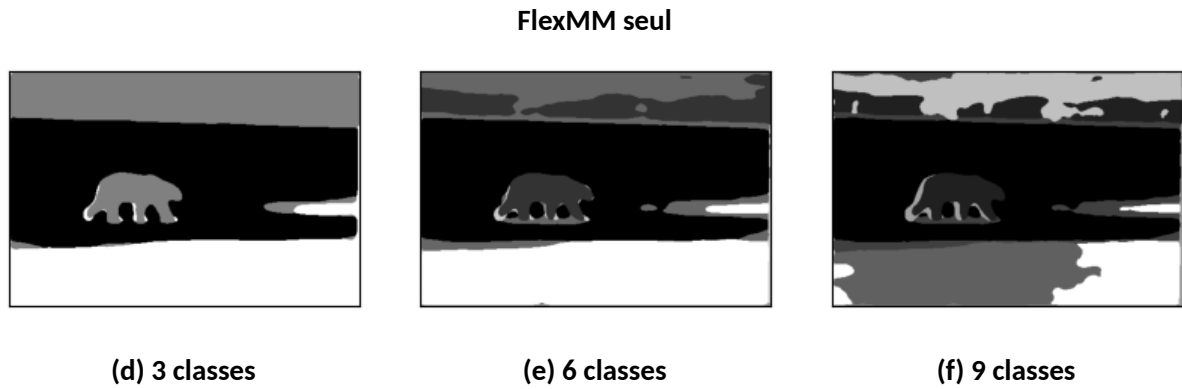


Fig. 3.13 : Segmentation avec FlexMM seul pour différents nombres de classes

Ces figures illustrent la segmentation des images obtenues avec SAM combiné à FlexMM (en haut) et avec FlexMM seul (en bas) pour différents nombres de classes.

Les métriques sont résumées dans le tableau ci-dessous.

Nb Classes	F-Score	ARI
3	0.40	0.92
6	0.47	0.92
9	0.42	0.95

Tab. 3.2 : SAM+FlexMM

Nb Classes	F-Score	ARI
3	0.40	0.91
6	0.34	0.91
9	0.12	0.89

Tab. 3.3 : FlexMM

3.2.3 Image encodée + FlexMM

La troisième idée creusée est d'utiliser juste le module encodeur de SAM pour extraire l'information de l'image, puis utiliser ces informations dans le modèle de mélange de lois. Mais nous nous sommes confrontés au problème lié à la taille de l'embedding donné par l'encodeur. Elle est plus petite que celle de l'image d'origine, ce qui fait qu'on obtient des segmentations qui sont trop suréchantillonnées. Une solution en cours d'exploration consiste à développer un encodeur produisant des caractéristiques à la même résolution que l'image d'origine, afin de limiter le suréchantillonnage observé dans les segmentations. Voici une segmentation suréchantillonnée trouvée en image.



Fig. 3.14 : Image originale



Fig. 3.15 : Image segmentée

Suite à ces différents résultats, on constate que des métriques comme le F-score et l'ARI ne corroborent pas toujours les résultats visuels, c'est-à-dire, qu'elles ne tiennent pas compte des recouvrements de masques ni de la qualité perceptuelle de la segmentation, ce qui incite à envisager le développement d'une nouvelle métrique mieux adaptée à ces spécificités.

3.3 Nouvelle métrique

Afin d'améliorer l'évaluation, plusieurs pistes de réflexion ont été explorées pour développer une métrique plus adaptée. Les idées de nouvelles métriques sont principalement concentrées sur l'utilisation de la carte de probabilité fournie par FlexMM. Une analyse de ses cartes de probabilité ont été effectuée par le biais du calcul de l'entropie par classe qui est définie par :

$$h_{ij} = - \sum_{k=1}^K p_{ij}(k) \log p_{ij}(k)$$

Avec :

- h_{ij} est l'entropie au pixel de position (i, j) ;
- K est le nombre de classes ou segments ;
- $p_{ij}(k)$ est la probabilité que le pixel (i, j) appartienne à la classe k ;
- Entropie moyenne H est la moyenne des h_{ij}

Dans un premier temps, il a été question de regarder l'évolution de l'entropie moyenne et le comportement de l'entropie au cours des itérations. Ce qui donne des résultats impressionnants. Il a été constaté que cette entropie moyenne diminue lorsque l'itération évolue, ce qui signifie que plus la segmentation est bonne, plus l'entropie moyenne est faible. Cela peut donc être utilisé comme un indicateur de performance ou d'appréciation des segmentations. Et ce sera encore plus intéressant si on arrive à construire des cartes de probabilité avec les vérités terrains. Voici un exemple d'évolution l'entropie moyenne :

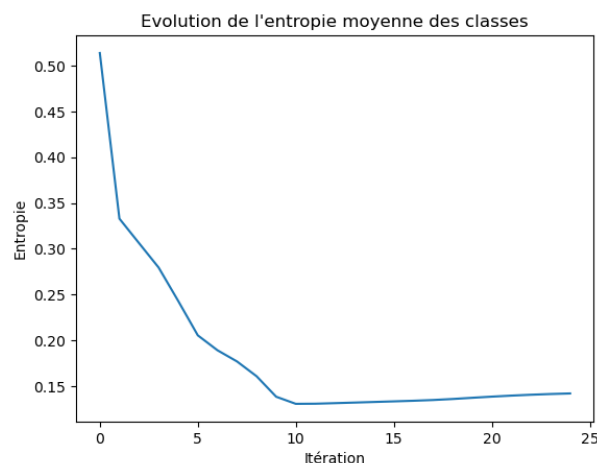


Fig. 3.16 : Exemple d'évolution de l'entropie moyenne

La possibilité de construire des cartes de probabilité à partir des vérités terrain a été explorée également, mais plusieurs difficultés ont été rencontrées pour la mise en place. Par exemple, les classes ne

sont pas ordonnées, toutes les vérités terrain n'ont pas le même nombre de classes. Néanmoins, une option a été creusée :

Pour chaque vérité terrain l , construire la matrice

$$M_l(k) = \begin{cases} 1 & \text{si le pixel } (i, j) \text{ appartient à la même classe } k, \\ 0 & \text{sinon.} \end{cases}$$

Ce qui produit une matrice de taille le carré de celle de l'image. En calculant :

$$P_k(i) = \frac{1}{J} \sum_{j=1}^J M_j(k) \quad \text{pour } k = 1, \dots, K$$

où J est le nombre total de vérités terrain disponible pour l'image.

On obtiendra des matrices qui peuvent être considérées comme la probabilité que deux pixels soient dans la classe k . Puis, à partir des P_k , $k = 1, \dots, K$, on calculera la probabilité pour qu'un pixel soit dans une classe en utilisant la formule de Bayes. Mais vu la taille de ces matrices, calculées avec un simple ordinateur est impossible. Cependant, des tests ont été effectués sur des petites portions d'images. (Voir annexe)

S'agissant du comportement de la carte d'entropie, on constate que l'entropie permet de délimiter les contours de la segmentation en prenant des valeurs élevées. En analysant cette carte au cours des itérations, on observe que plus on tend vers une meilleure segmentation, plus cette carte dessine clairement les contours.

Cela suggère que l'entropie peut être utilisée comme un bon indicateur de la précision et de la netteté des délimitations entre les différentes régions segmentées. Voici un exemple de l'évolution de la carte d'entropie.

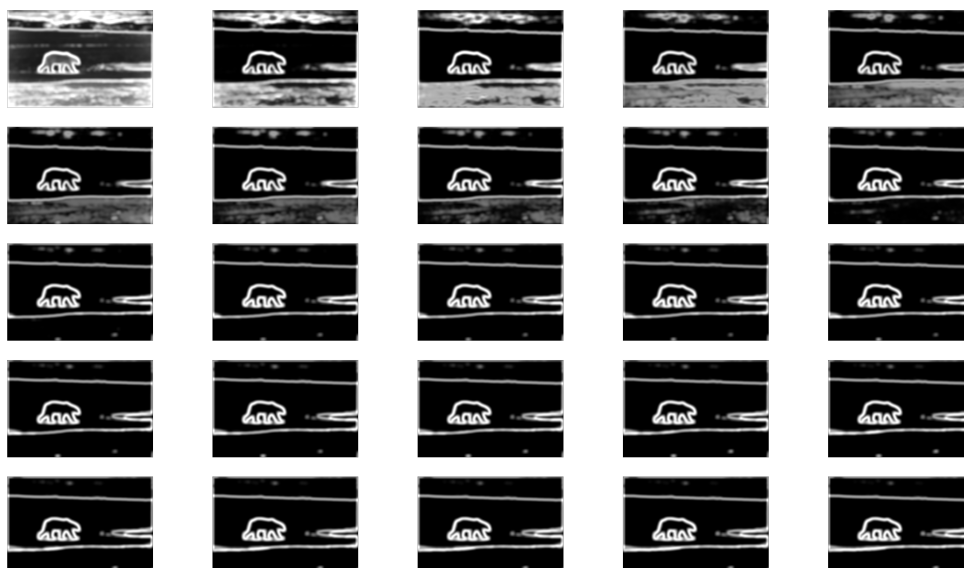


Fig. 3.17 : Exemple d'évolution de la carte d'entropie

En résumé, l'entropie peut être utilisée comme un indicateur car elle reflète le degré d'incertitude

ou de confiance du modèle dans la segmentation. Plus l'entropie est faible, plus la segmentation est déterminée (donc potentiellement meilleure), tandis qu'une entropie élevée indique une ambiguïté dans l'affectation des pixels aux classes, ce qui peut signaler des erreurs ou des zones mal segmentées. L'analyse de l'évolution de la carte d'entropie permet de constater qu'elle permet de quantifier l'incertitude du modèle sur la classification des pixels. Car, au début de l'algorithme, cette incertitude est élevée non seulement autour des contours des objets, mais aussi dans les zones texturées de l'image, où les caractéristiques sont plus complexes et ambiguës. Au fur et à mesure de l'optimisation, l'incertitude diminue dans les régions texturées et reste principalement localisée aux contours, où la distinction entre classes est naturellement plus difficile. Ces résultats sont cohérents avec nos intuitions sur la segmentation. Ce qu'on aimerait retrouver également en analysant les vérités terrains pour confirmer l'utilisation de l'entropie comme métrique d'évaluation.

3.4 Discussion et suggestions

3.4.1 Discussion

Les expériences ont montré que la combinaison SAM + FlexMM permet généralement d'obtenir de meilleures métriques de segmentation que l'utilisation de FlexMM seul. Les scores de F-Score et d'ARI sont globalement plus élevés, indiquant une meilleure cohérence entre les masques générés et les classes cibles.

Cependant, on constate que cette amélioration métrique ne se traduit pas toujours par une segmentation visuellement satisfaisante. Certains objets sont mal séparés ou incorrectement détectés. Cela peut s'expliquer par le fait que SAM génère des masques avec chevauchement, c'est-à-dire qu'un même pixel peut appartenir à plusieurs masques. Cette propriété peut perturber l'étape de classification ou d'affectation finale par FlexMM.

De plus, lorsque SAM est appliqué après FlexMM, les résultats visuels sont légèrement proches de celui trouvé lorsqu'on applique SAM seul, malgré le f-score décroît considérablement et que le nombre de région double. Ce constat met en lumière une limitation des métriques classiques, comme l'ARI, qui ne prend pas toujours en compte la qualité perçue par l'utilisateur. Ce qui peut être dû au fait que les classes ne sont pas ordonnées et le nombres de classes entre les vérités terrain et les segmentations ne sont pas identiques.

3.4.2 Suggestions

Sur la base de ces résultats, plusieurs pistes de réflexion peuvent être envisagées afin d'améliorer la qualité globale de la segmentation et d'adresser les limites observées :

- Mettre en place un nettoyage ou un filtrage des masques générés par SAM avant leur utilisation, afin d'éliminer les régions parasites ou peu pertinentes ;
- Introduire des règles d'exclusion systématiques permettant de gérer les chevauchements de pixels entre masques, pour garantir une meilleure séparation des objets segmentés ;
- Explorer des approches hybrides qui combinent la segmentation avec la détection d'objet, dans le but d'isoler plus précisément les régions d'intérêt et d'améliorer la cohérence spatiale des

masques.

Par ailleurs, la réalisation de tests sur des jeux de données plus variés et finement annotés serait essentielle pour affiner les paramètres du modèle, valider la généralisabilité des résultats, et évaluer la robustesse de l'approche face à des scénarios complexes et diversifiés. Ces expérimentations permettront également de mieux cerner les conditions dans lesquelles les performances peuvent être optimisées.

CONCLUSION

La segmentation d'images est un problème mal posé dans la mesure où la segmentation idéale n'est généralement pas connue. Toutefois, en y intégrant des contraintes de régularité, il devient possible d'obtenir des solutions cohérentes et exploitables. Plusieurs techniques différentes permettent de faire une segmentation perceptuelle d'images. Les techniques basées sur les mélanges de lois ont prouvé leur efficacité. Leur objectif est d'estimer la loi de probabilité multimodale à laquelle appartient chaque pixel. Elles offrent ainsi un cadre pertinent pour quantifier l'incertitude tout au long du processus de segmentation. Cependant, ces méthodes sont confrontées à plusieurs défis, notamment le choix du nombre optimal de classes.

D'un autre côté, les méthodes basées sur l'apprentissage profond, comme le modèle SAM, fonctionnent davantage comme des boîtes noires, il est difficile d'expliquer précisément comment la segmentation est réalisée. Néanmoins, leur capacité à détecter automatiquement des objets par génération de masques est indéniable. Cette génération se déroule en trois étapes : l'encodage de l'image, l'encodage du *prompt*, et la production du masque. Bien que très performantes, ces méthodes peuvent conduire à une sur-segmentation, ce qui les rend surtout adaptées à la détection d'objets.

L'association de ces deux approches probabilistes et neuronales peut potentiellement produire des résultats intéressants. Toutefois, un problème majeur demeure. En segmentation perceptuelle, les classes ne sont généralement pas ordonnées, ce qui complique le choix de métriques adaptées pour évaluer les résultats. Pour obtenir une segmentation cohérente, il est essentiel que l'initialisation respecte une certaine méthodologie. Nos travaux ont montré que les métriques classiques comme le F-score et l'ARI ne sont pas toujours pertinentes dans ce contexte. L'entropie, quant à elle, se révèle plus prometteuse pour évaluer la qualité des segmentations probabilistes, mais son utilisation reste limitée par des difficultés liées à son calcul sur les vérités terrain. Cela soulève une question centrale : quelles nouvelles métriques pourraient mieux capter la cohérence perceptuelle d'une segmentation ?

Pour notre application, les résultats sont satisfaisants pour les images naturelles. Les applications ont été réalisées en python. Ce travail a permis de gagner énormément en compétences théoriques et pratiques, tant sur les méthodes probabilistes que sur les approches par apprentissage profond. En résumé, cette expérience a enrichi notre compréhension des enjeux de la segmentation d'images et

nous a donné de solides bases pour de futures recherches dans le domaine.

Enfin, une piste d'amélioration serait de développer une métrique hybride combinant incertitude, cohérence des contours et distance aux vérités terrain, afin de mieux évaluer les segmentations complexes. D'autres travaux pourraient aussi explorer l'utilisation de **prompts** spécifiques dans SAM pour améliorer la précision des masques produits. Les méthodes pourraient être testées sur différents types d'images telles que l'imagerie médicale.

GLOSSAIRE

SAM	Segment Anything Model
FlexMM	Flexible Mixture Model
GMM	Gaussian Mixture Model
sGMM	Smoothed Gaussian Mixture Model
SMM	Student Mixture Model
sSMM	Smoothed Student Mixture Model
ARI	Adjusted Rand Index

BIBLIOGRAPHIE

- [1] Gonzalez, Rafael C. et Woods, Richard E. *Digital Image Processing*. 4th. Chapitre 1, page 18. Boston : Pearson, 2008.
- [2] McLachlan, Geoffrey J., Lee, Sharon X. et Rathnayake, Suren I. "Finite Mixture Models". In : *Annual Review of Statistics and Its Application* 6 (2019). First published online September 13, 2018, p. 355-378. doi : 10.1146/annurev-statistics-031017-100325. url : <https://snunnari.github.io/SBE/mclachlan.pdf>.
- [3] Vacher, Jonathan, Launay, Claire et Coen-Cagli, Ruben. "Flexibly Regularized Mixture Models and Application to Image Segmentation". In : *Neurocomputing* 468 (2022), p. 1-15. doi : 10.1016/j.neucom.2022.02.009. url : <https://doi.org/10.1016/j.neucom.2022.02.009>.
- [4] Kirillov, Alexander et al. "Segment Anything". In : *arXiv* 2304.02643 (2023). arXiv :2304.02643v1 [cs.CV] 5 Apr 2023, p. 1-23. url : <https://arxiv.org/pdf/2304.02643>.
- [5] Badrinarayanan, Vijay, Kendall, Alex et Cipolla, Roberto. "SegNet : A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation". In : *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 39.12 (2017), p. 2481-2495. doi : 10.1109/TPAMI.2016.2644615. url : <https://doi.org/10.1109/TPAMI.2016.2644615>.
- [6] Ronneberger, Olaf, Fischer, Philipp et Brox, Thomas. "U-Net : Convolutional Networks for Biomedical Image Segmentation". In : *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Available online. Munich, Germany : Springer, 2015, p. 234-241. url : <http://lmb.informatik.uni-freiburg.de/Publications/2015/RFB15a/>.
- [7] Wikipédia. *Segmentation d'images*. [Consulté le 30 juillet 2025]. 2025. url : https://fr.wikipedia.org/wiki/Segmentation_d'image.
- [8] Kermad, Chafik. "Segmentation d'image : recherche d'une mise en œuvre automatique par coopération de méthodes". français. Traitement du signal et de l'image [eess.SP]. Thèse de doctorat. Université Rennes 1, 1997. url : <https://theses.hal.science/tel-00008781>.
- [9] Gonzalez, Rafael C. et Woods, Richard E. *Digital Image Processing*. 4^e éd. p. 700, Chapter 10 : Image Segmentation. Boston : Pearson, 2018.

-
- [10] Kass, Michael, Witkin, Andrew et Terzopoulos, Demetri. "Snakes : Active Contour Models". In : *International Journal of Computer Vision* 1.4 (1988), p. 321-331. doi : 10 . 1007 / BF00133570. url : <https://doi.org/10.1007/BF00133570>.
 - [11] Coen-Cagli, R., Kohn, A. et Schwartz, O. "Flexible gating of contextual influences in natural vision". In : *Nature Neuroscience* 12.11 (2009), p. 1479-1481. doi : 10 . 1038/nn . 2415.
 - [12] Wainwright, M. J. et Simoncelli, E. P. "Scale mixtures of Gaussians and the statistics of natural images". In : *Advances in Neural Information Processing Systems* 12 (2000), p. 855-861.
 - [13] Sun, Min et al. "Learning structured output representation using deep conditional generative models". In : *Proceedings of the 34th International Conference on Machine Learning* (2017), p. 3267-3276.
 - [14] Nikou, Christophoros, Galatsanos, Nikolas P. et Likas, Aristidis. "A mixture model for image segmentation using spatial information with application to medical images". In : *IEEE Transactions on Image Processing* 19.3 (2010), p. 733-745.
 - [15] Koller, Daphne et Friedman, Nir. *Probabilistic Graphical Models : Principles and Techniques*. Cambridge, MA : MIT Press, 2009. isbn : 978-0-262-01319-2.
 - [16] Dempster, Arthur P, Laird, Nan M et Rubin, Donald B. "Maximum likelihood from incomplete data via the EM algorithm". In : *Journal of the Royal Statistical Society : Series B (Methodological)* 39.1 (1977), p. 1-22.

ANNEXE

Calcule des mises à jour du modèle de mélange simple

- C_n variable latente
- $\mathbf{p} = (p_k)_{k=1}^K$ avec $p_k = P_{C_n}(k)$
- $P_{X|\mathbf{p},\mathbf{a}}(X|\mathbf{p},\mathbf{a}) = \sum_{k=1}^K p_k P_{X^{(k)}}(X; a_k)$
- La log-vraisemblance : $\ell(\mathbf{p}, \mathbf{a}; (x_n, C_n)_n) = \ln \left(\prod_{n=1}^N P_{X,C|\mathbf{p},\mathbf{a}}(x_n, C_n|\mathbf{p}, \mathbf{a}) \right)$

En remplaçant par la distribution conditionnelle :

$$\ell(\mathbf{p}, \mathbf{a}; (x_n, C_n)_n) = \ln \left(\prod_{n=1}^N \prod_{k=1}^K \left(p_k P_X^{(k)}(x_n; a_k) \right)^{\mathbf{1}_k(C_n)} \right)$$

où

$$\mathbf{1}_j(i) = \begin{cases} 1, & \text{si } i = j \\ 0, & \text{sinon.} \end{cases}$$

$$\ell(\mathbf{p}, \mathbf{a}; (x_n, C_n)_n) = \sum_{n=1}^N \sum_{k=1}^K \mathbf{1}_k(C_n) \ln p_k + \mathbf{1}_k(C_n) \ln P_X^{(k)}(x_n; a_k)$$

L'algorithme EM :

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) = \mathbb{E}_{C_n|(X_n)_n, \boldsymbol{\theta}} \left[\ell(\mathbf{p}, \mathbf{a}; (x_n, C_n)_n) \mid (x_n)_n, \boldsymbol{\theta}^{(t)} \right]$$

En pratique, l'étape E consiste à calculer pour tout $n \in \{1, \dots, N\}$ et $k \in \{1, \dots, K\}$:

$$\tau_{n,k}^{(t)} = P_{(C_n|X_n, \boldsymbol{\theta})}(k|x_n, \boldsymbol{\theta}^{(t)}) = \mathbb{E}_{C_n|X_n, \boldsymbol{\theta}}[\mathbf{1}_k(C_n) \mid x_n, \boldsymbol{\theta}^{(t)}]$$

Enfin, l'étape M met à jour les paramètres en maximisant Q :

$$\boldsymbol{\theta}^{(t+1)} = \arg \max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)})$$

Calculons l'espérance :

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) = \mathbb{E}_{C_n | (X_n)_n, \boldsymbol{\theta}} \left[\ell(\mathbf{p}, \mathbf{a}; (x_n, C_n)_n) \mid (x_n)_n, \boldsymbol{\theta}^{(t)} \right]$$

En remplaçant la log-vraisemblance $\ell(\mathbf{p}, \mathbf{a}; (x_n, C_n)_n)$ par son expression et en prenant l'espérance conditionnelle sous $C_n \mid X_n, \boldsymbol{\theta}^{(t)}$, on obtient :

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) = \sum_{n=1}^N \sum_{k=1}^K \mathbb{E}_{C_n | X_n, \boldsymbol{\theta}^{(t)}} [\mathbf{1}_k(C_n)] \left(\ln p_k + \ln P_X^{(k)}(x_n; a_k) \right)$$

Par définition de l'espérance d'une variable indicatrice :

$$\mathbb{E}_{C_n | X_n, \boldsymbol{\theta}^{(t)}} [\mathbf{1}_k(C_n)] = P(C_n = k | X_n, \boldsymbol{\theta}^{(t)}) = \tau_{n,k}^{(t)}$$

Nous obtenons alors :

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) = \sum_{n=1}^N \sum_{k=1}^K \tau_{n,k}^{(t)} \left(\ln p_k + \ln P_X^{(k)}(x_n; a_k) \right)$$

Déterminons à présent $\boldsymbol{\theta}^{(t+1)}$

Sachant que :

$$\boldsymbol{\theta} = (\mathbf{p}, \mathbf{a}) = (p_1, p_2, \dots, p_K, a_1, a_2, \dots, a_K)$$

L'optimisation de $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)})$ peut se faire en maximisant séparément les paramètres $\mathbf{p}$ et $\mathbf{a}$.

L'expression de $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)})$ est :

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) = \sum_{n=1}^N \sum_{k=1}^K \tau_{n,k}^{(t)} \ln(p_k) + \sum_{n=1}^N \sum_{k=1}^K \tau_{n,k}^{(t)} \ln P_X^{(k)}(x_n; a_k)$$

1. Maximisation par rapport à $\mathbf{p}$:

Les p_k sont des probabilités et doivent satisfaire la contrainte $\sum_{k=1}^K p_k = 1$.

On considère donc le lagrangien :

$$\mathcal{L}(\mathbf{p}, \lambda) = \sum_{n=1}^N \sum_{k=1}^K \tau_{n,k}^{(t)} \ln(p_k) + \lambda \left(1 - \sum_{k=1}^K p_k \right)$$

On dérive par rapport à p_k :

$$\frac{\partial \mathcal{L}}{\partial p_k} = \sum_{n=1}^N \frac{\tau_{n,k}^{(t)}}{p_k} - \lambda = 0, \quad k = 1, \dots, K \quad (1)$$

On somme sur k et on utilise la contrainte $\sum_{k=1}^K p_k = 1$:

$$\sum_{k=1}^K p_k = \frac{1}{\lambda} \sum_{k=1}^K \sum_{n=1}^N \tau_{n,k}^{(t)} = 1$$

Donc,

$$\lambda = N$$

Finalement, en remplaçant λ dans l'équation (1), la mise à jour est :

$$p_k^{(t+1)} = \frac{\sum_{n=1}^N \tau_{n,k}^{(t)}}{N}$$

Puis que

$$\frac{\partial^2 \mathcal{L}}{\partial p_k^2} = - \sum_{n=1}^N \frac{\tau_{n,k}^{(t)}}{p_k^2} < 0$$

2. Maximisation par rapport à a : L'optimisation de a_k dépend de la forme de $P_{X^{(k)}}(x_n; a_k)$. En général, on doit résoudre :

$$a_k^{(t+1)} = \arg \max_{a_k} \sum_{n=1}^N \tau_{n,k}^{(t)} \ln P_{X^{(k)}}(x_n; a_k)$$

- Si $P_{X^{(k)}}$ est une loi normale $\mathcal{N}(\mu_k, \Sigma_k)$

La log-vraisemblance pour un ensemble de N observations $x_1, x_2, \dots, x_N$ est :

$$\ln P_{X^{(k)}}(x_n; a_k) = -\frac{d}{2} \ln(2\pi) - \frac{1}{2} \ln |\Sigma_k| - \frac{1}{2} (x_n - \mu_k)^T \Sigma_k^{-1} (x_n - \mu_k)$$

où d est la dimension de x_n .

Posons

$$J = \sum_{n=1}^N \tau_{n,k}^{(t)} \ln P_{X^{(k)}}(x_n; a_k)$$

Dérivée par rapport à μ_k :

$$\frac{\partial J}{\partial \mu_k} = \sum_{n=1}^N \tau_{n,k}^{(t)} \left[-\frac{\nu_k + d}{2} \frac{2 \Sigma_k^{-1} (x_n - \mu_k)}{\nu_k + \delta_{n,k}} \right]$$

En posant $w_{n,k} = \frac{\nu_k + d}{\nu_k + \delta_{n,k}}$, on obtient l'équation suivante :

$$\sum_{n=1}^N \tau_{n,k}^{(t)} w_{n,k} \Sigma_k^{-1} (x_n - \mu_k) = 0$$

Ce qui donne l'estimateur de μ_k :

$$\mu_k^{(t+1)} = \frac{\sum_{n=1}^N \tau_n^{(t)}(k) w_{n,k} x_n}{\sum_{n=1}^N \tau_n^{(t)}(k) w_{n,k}}$$

Dérivée par rapport à Σ_k :

En appliquant les formules suivantes :

$$\frac{\partial}{\partial X} (a^T X^{-1} b) = -X^{-T} a b^T X^{-T}$$

$$\frac{\partial}{\partial X} \ln(\det(X^{-1})) = X^{-1}$$

avec $X = \Sigma_k$, $a = x_n - \mu_k$ et $b = x_n - \mu_k$, et en utilisant le fait que Σ_k est symétrique, On obtient :
La dérivée par rapport à Σ_k est :

$$\frac{\partial J}{\partial \Sigma_k} = \sum_{n=1}^N \tau_{n,k}^{(t)} \left[-\frac{1}{2} \Sigma_k^{-1} + \frac{\nu_k + d}{2} \frac{\Sigma_k^{-1} (x_n - \mu_k)(x_n - \mu_k)^T \Sigma_k^{-1}}{\nu_k + \delta_{n,k}} \right]$$

En réarrangeant cette expression, on obtient :

$$\Sigma_k^{-1} \left(\sum_{n=1}^N w_{n,k} (x_n - \mu_k)(x_n - \mu_k)^T - \left(\sum_{n=1}^N \tau_{n,k}^{(t)} \Sigma_k \right) \Sigma_k^{-1} \right) = 0$$

On en déduit l'estimateur de Σ_k :

$$\Sigma_k^{(t+1)} = \frac{\sum_{n=1}^N \tau_{n,k}^{(t)} w_{n,k} (x_n - \mu_k^{(t+1)})(x_n - \mu_k^{(t+1)})^T}{\sum_{n=1}^N \tau_{n,k}^{(t)}}$$

Dérivée par rapport à ν_k

Posons :

$$\psi(x) = \frac{d}{dx} \ln \Gamma(x) = \frac{\Gamma'(x)}{\Gamma(x)}$$

- Premier terme : $\ln \Gamma\left(\frac{\nu_k + d}{2}\right)$

$$\frac{\partial}{\partial \nu_k} \ln \Gamma\left(\frac{\nu_k + d}{2}\right) = \frac{1}{2} \psi\left(\frac{\nu_k + d}{2}\right)$$

- Deuxième terme : $-\ln \Gamma\left(\frac{\nu_k}{2}\right)$

$$\frac{\partial}{\partial \nu_k} \left(-\ln \Gamma\left(\frac{\nu_k}{2}\right) \right) = -\frac{1}{2} \psi\left(\frac{\nu_k}{2}\right)$$

- Troisième terme : $-\frac{d}{2} \ln(\nu_k \pi)$

$$\frac{\partial}{\partial \nu_k} \left(-\frac{d}{2} \ln(\nu_k \pi) \right) = -\frac{d}{2\nu_k}$$

- Quatrième terme : $-\frac{1}{2} \ln |\Sigma_k|$

Ce terme ne dépend pas de ν_k , donc sa dérivée est nulle.

- Cinquième terme : $-\frac{\nu_k + d}{2} \ln \left(1 + \frac{1}{\nu_k} \delta_n \right)$

Nous appliquons la dérivation de la fonction logarithme :

$$\frac{\partial}{\partial \nu_k} \left(-\frac{\nu_k + d}{2} \ln \left(1 + \frac{1}{\nu_k} \delta_n \right) \right) = -\frac{1}{2} \left(\ln \left(1 + \frac{1}{\nu_k} \delta_n \right) + \frac{\nu_k + d}{\nu_k^2 + \nu_k \delta_n} \delta_n \right)$$

- Dérivée totale :

En combinant les dérivées des termes, nous obtenons l'expression suivante pour la dérivée de $\ln P_X^{(k)}(x_n, a_k)$ par rapport à ν_k :

$$\frac{\partial}{\partial \nu_k} \ln P_X^{(k)}(x_n, a_k) = \frac{1}{2} \psi \left(\frac{\nu_k + d}{2} \right) - \frac{1}{2} \psi \left(\frac{\nu_k}{2} \right) - \frac{d}{2\nu_k} - \frac{1}{2} \left(\ln \left(1 + \frac{1}{\nu_k} \delta_n \right) + \frac{\nu_k + d}{\nu_k^2 + \nu_k \delta_n} \delta_n \right)$$

Ici on considère que l'indice du pixel influence également la probabilité et donc on a :

$$P_{X_n|P_n,A}(x|p_n, a) = \sum_{k=1}^K p_{n,k} P_{X(k)}(x; a_k)$$

- **Distribution Gamma** : Soit R une v.a.r $R \sim G(S)$.

$$P_{R|S}(r|s) = \frac{r^{s-1} \exp(-r)}{\Gamma(s)}, \quad r \geq 0, s > 0.$$

- **Distribution de Diriclet** : Soit R une v.a.r $R \sim D(S)$.

$$P_{R|S}(r|s) = \frac{\Gamma \left(\sum_{k=1}^K s_k \right)}{\prod_{k=1}^K \Gamma(s_k)} \prod_{k=1}^K r_k^{s_k-1}, \quad r_k \geq 0, \sum_{k=1}^K r_k = 1.$$

Preuve : Proposition 2.1.1

Le log-postérieur des paramètres p et a est donné par :

$$\ell(\mathbf{p}, \mathbf{a}; (x_n, C_n, B_n)_n) = \ln \left(P_{P,A|X,C,B}(\mathbf{p}, \mathbf{a} | (x_n)_n, (C_n)_n, (B_n)_n) \right).$$

$$P_{P,A|(x_n)_n,C,B}(p, a | (x_n)_n, C, B) = P((x_n)_n | C_n, a) P(C_n | p_n) P(p_n | B_n) P(\mathbf{p}, \mathbf{a})$$

Loi a priori Dirichlet sur p_n :

$$P(p_n | B_n) \propto \prod_{k=1}^K p_{n,k}^{B_{n,k}-1} \Rightarrow \ln P(p_n | B_n) = \sum_{k=1}^K (B_{n,k} - 1) \ln p_{n,k} + \text{const.}$$

Vraisemblance de C_n sachant p_n :

$$P(C_n = k | p_n) = p_{n,k} \Rightarrow \ln P(C_n | p_n) = \sum_{k=1}^K \mathbf{1}_k(C_n) \ln p_{n,k}.$$

Autres termes ($P((x_n)_n | C_n, a)$, $P(a)$, etc.) $\rightarrow$ regroupés dans $W(\cdot)$.

On additionne les contributions de $P(p_n | B_n)$ et $P(C_n | p_n)$:

$$\sum_{k=1}^K [B_{n,k} - 1 + \mathbf{1}_k(C_n)] \ln p_{n,k}.$$

Et enfin, on obtient :

$$\ell(\mathbf{p}, \mathbf{a}; (x_n, C_n, B_n)_n) = \sum_{n=1}^N \sum_{k=1}^K (B_{n,k} - 1 + \mathbf{1}_k(C_n)) \ln p_{n,k} + W((x_n)_n, C, B; \alpha).$$

Connaissant l'estimation précédente des paramètres $\theta^{(t)}$, l'étape E consiste à prendre l'espérance conditionnelle du logarithme de la postérieure ℓ , soit :

$$Q(\theta; \theta^{(t)}, (x_n)_n) = \sum_{n=1}^N \sum_{k=1}^K \mathbb{E}[B_{n,k} - 1 + \mathbf{1}_k(C_n) \mid (x_n)_n, \theta^{(t)}] \ln(p_{n,k}) + w((x_n)_n; \alpha, \theta^{(t)}),$$

avec

$$w((x_n)_n; \alpha, \theta^{(t)}) = \mathbb{E}[W((x_n, C_n, B_n)_n; \alpha) \mid (x_n)_n, \theta^{(t)}].$$

Or

$$\mathbb{E}[B_{n,k} - 1 + \mathbf{1}_k(C_n) \mid (x_n)_n, \theta^{(t)}] = \mathbb{E}[u_{n,k}(\mathbf{1}_k(C_n)) \mid (x_n)_n, \theta^{(t)}] = u_{n,k} \mathbb{E}[\mathbf{1}_k(C_n) \mid (x_n)_n, \theta^{(t)}] = u_{n,k}(\tau_{\cdot,k}^{(t)}),$$

D'où :

$$Q(\theta, \theta^{(t)}) = \sum_{n=1}^N \sum_{k=1}^K u_{n,k}(\tau_{\cdot,k}^{(t)}) \ln p_{n,k} + w((x_n)_n; \alpha, \theta^{(t)})$$

Pour obtenir la règle de mise à jour de p_n , on ajoute un multiplicateur de Lagrange associé à la contrainte $\sum_k p_{n,k} = 1$ et on calcule la dérivée partielle par rapport à $p_{n,k}$. On obtient donc, pour tout $(n, k) \in \{1, \dots, N\} \times \{1, \dots, K\}$:

$$\frac{u_{n,k}(\tau_{\cdot,k}^{(t)})}{p_{n,k}} + \lambda_n = 0,$$

ce qui conduit à la règle de mise à jour en choisissant λ_n tel que $\sum_k p_{n,k} = 1$.

Preuve : Proposition 2.1.2

On note θ les paramètres du modèle $(a, (p_n)_n)$. À l'itération $t + 1$, l'étape E calcule :

$$\begin{aligned} Q(\theta; \theta^{(t)}, (x_n)_n) &= \mathbb{E}_{C,B \mid X, \theta^{(t)}} [\ln P_{\theta \mid X, C, B}(\theta \mid (x_n)_n, (C_n)_n, (B_n)_n)] \\ &= \mathbb{E}_{C,B \mid X, \theta^{(t)}} [\ln P_{X, C, B, \theta}((x_n)_n, (C_n)_n, (B_n)_n, \theta) \\ &\quad - \ln P_{X, C, B}((x_n)_n, (C_n)_n, (B_n)_n)]. \end{aligned}$$

L'étape M maximise ensuite $Q(\theta; \theta^{(t)}, (x_n)_n)$ par rapport à θ .

En utilisant la règle de la chaîne :

$$P((x_n)_n, (C_n)_n, (B_n)_n, \theta) = P((C_n)_n, (B_n)_n \mid (x_n)_n, \theta) P(\theta \mid (x_n)_n) P((x_n)_n).$$

Ainsi, pour tout θ :

$$P((x_n)_n, (C_n)_n \mid \theta) = P((C_n)_n \mid (x_n)_n, \theta) P((x_n)_n \mid \theta),$$

et donc :

$$\ln P((x_n)_n \mid \theta) = \ln P((x_n)_n, (C_n)_n \mid \theta) - \ln P((C_n)_n \mid (x_n)_n, \theta).$$

En prenant l'espérance par rapport à $C \mid X, \theta^{(t)}$:

$$\mathbb{E}_{C \mid X, \theta^{(t)}} [\ln P((x_n)_n \mid \theta)] = Q(\theta; \theta^{(t)}, (x_n)_n) + H((C_n)_n \mid (x_n)_n, \theta^{(t)}, (C_n)_n \mid (x_n)_n, \theta),$$

où $H(p, q)$ désigne l'entropie croisée des distributions p et q .

En soustrayant pour $\theta = \theta^{(t)}$:

$$\ln P((x_n)_n \mid \theta) - \ln P((x_n)_n \mid \theta^{(t)}) = Q(\theta; \theta^{(t)}, (x_n)_n) - Q(\theta^{(t)}; \theta^{(t)}, (x_n)_n) + H(\dots) - H(\dots).$$

D'après l'inégalité de Gibbs $H(p, q) \geq H(p, p)$:

$$\ln P((x_n)_n \mid \theta) - \ln P((x_n)_n \mid \theta^{(t)}) \geq Q(\theta; \theta^{(t)}, (x_n)_n) - Q(\theta^{(t)}; \theta^{(t)}, (x_n)_n).$$

Application au log-postérieur. On écrit :

$$\ln P(\theta \mid (x_n)_n) = \ln P((x_n)_n, (C_n)_n, (B_n)_n, \theta) - \ln P((C_n)_n, (B_n)_n \mid (x_n)_n, \theta) - \ln P((x_n)_n).$$

En prenant l'espérance et en soustrayant pour $\theta = \theta^{(t)}$, on obtient :

$$\ln P(\theta \mid (x_n)_n) - \ln P(\theta^{(t)} \mid (x_n)_n) > Q(\theta; \theta^{(t)}, (x_n)_n) - Q(\theta^{(t)}; \theta^{(t)}, (x_n)_n).$$

Comme l'étape M définit $\theta^{(t+1)}$ de façon à maximiser $Q(\theta; \theta^{(t)}, (x_n)_n)$, on conclut :

$$\ln P(\theta^{(t+1)} \mid (x_n)_n) - \ln P(\theta^{(t)} \mid (x_n)_n) > 0.$$

Preuve : Proposition 2.1.3

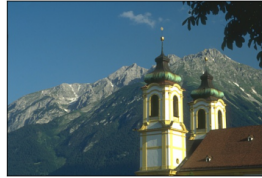
Cette preuve est similaire à celle de la proposition 2.1.1 juste qu'ici, il faut utiliser :

$$\mathbb{E}[B_{n,k}^{(h)} - 1 + \mathbf{1}_k(C_n^{(h)}) \mid (x_n)_n, \theta^{(t)}] = \mathbb{E}[u_{n,k}^{(h)} \mathbf{1}_k(C^{(1)}), \dots, \mathbf{1}_k(C^{(H)}) \mid (x_n^{(h)})_n, \theta^{(t,h)}].$$

$$= u_{n,k}^{(h)} \mathbb{E}[\mathbf{1}_k(C^{(1)}), \dots, \mathbf{1}_k(C^{(H)}) \mid (x_n^{(h)})_n, \theta^{(t,h)}] = u_{n,k}^{(h)} \left(\tau_{\cdot,k}^{(t,1)}, \dots, \tau_{\cdot,k}^{(t,H)} \right).$$

Résultats de l'article FlexMM

Ici, nous présentons fidèlement les résultats de l'article que nous avons reproduits.



On nommera respectivement 1, 2, 3 et 4 ces images de gauche vers la droite.

Tab. 3.4 : Résultats aRI pour chaque image, modèle et valeur de K

Image	Modèle	$K = 3$	$K = 6$	$K = 9$
1	sSMM	0.44	0.65	0.61
	SMM	0.25	0.59	0.5
	sGMM	0.22	0.51	0.43
	GMM	0.23	0.60	0.64
2	sSMM	0.50	0.71	0.55
	SMM	0.49	0.68	0.50
	sGMM	0.43	0.31	0.40
	GMM	0.39	0.64	0.47
3	sSMM	0.18	0.1	0.07
	SMM	0.10	0.01	0.05
	sGMM	0.00	0.00	0.00
	GMM	0.22	0.12	0.09
4	sSMM	0.16	0.04	0.03
	SMM	0.11	0.05	0.02
	sGMM	0.00	0.00	0.00
	GMM	0.19	0.11	0.05

Image	Modèle	$K = 3$	$K = 6$	$K = 9$
1	sMM	0.79	0.8	0.85
	SMM	0.7	0.8	0.77
	sGMM	0.51	0.72	0.69
	GMM	0.80	0.84	0.8
2	sSMM	0.70	0.72	0.68
	SMM	0.70	0.72	0.67
	sGMM	0.58	0.5	0.54
	GMM	0.66	0.64	0.63
3	sSMM	0.36	0.28	0.29
	SMM	0.26	0.27	0.28
	sGMM	0.00	0.09	0.14
	GMM	0.30	0.29	0.27
4	sSMM	0.53	0.28	0.24
	SMM	0.40	0.26	0.24
	sGMM	0.06	0.06	0.08
	GMM	0.34	0.31	0.23

Tab. 3.5 : Résultats F-score pour chaque image, modèle et valeur de K

Les deux tableaux montrent que le modèle sSMM (Student Mixture Model avec lissage spatial) est globalement le plus performant et le plus robuste. Sur les images 1 et 2, les scores aRI et F-score sont élevés, notamment pour $K = 6$ et $K = 9$, confirmant l'intérêt du lissage spatial et de la loi de Student-t pour gérer les valeurs aberrantes. En revanche, pour les images 3 et 4, les performances chutent fortement pour tous les modèles, avec des valeurs proches de zéro, ce qui traduit la difficulté de segmentation dans des contextes complexes.

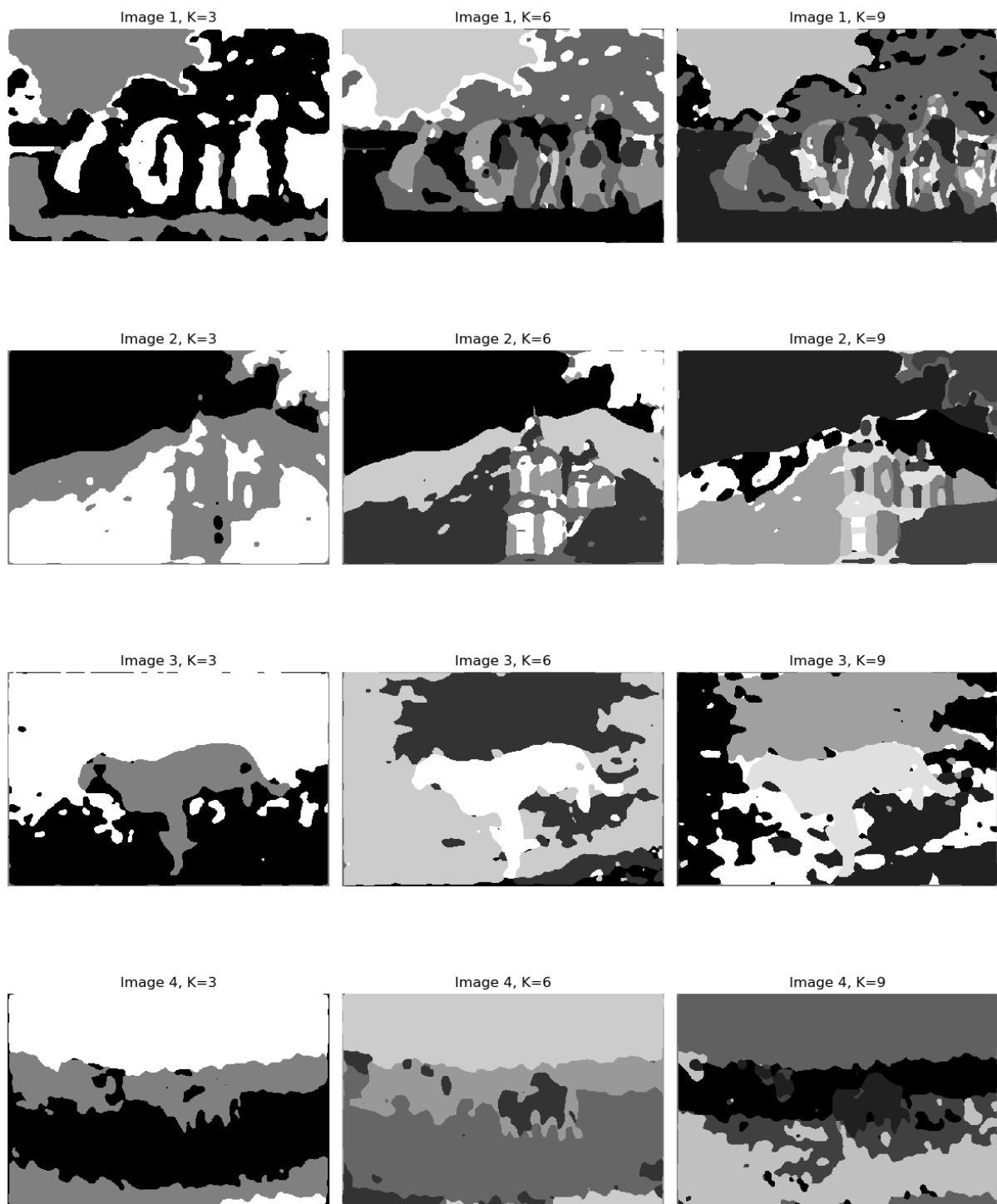


Fig. 3.18 : Le modèle SMM(mélange utilisant des distributions de Student-t sans lissage)

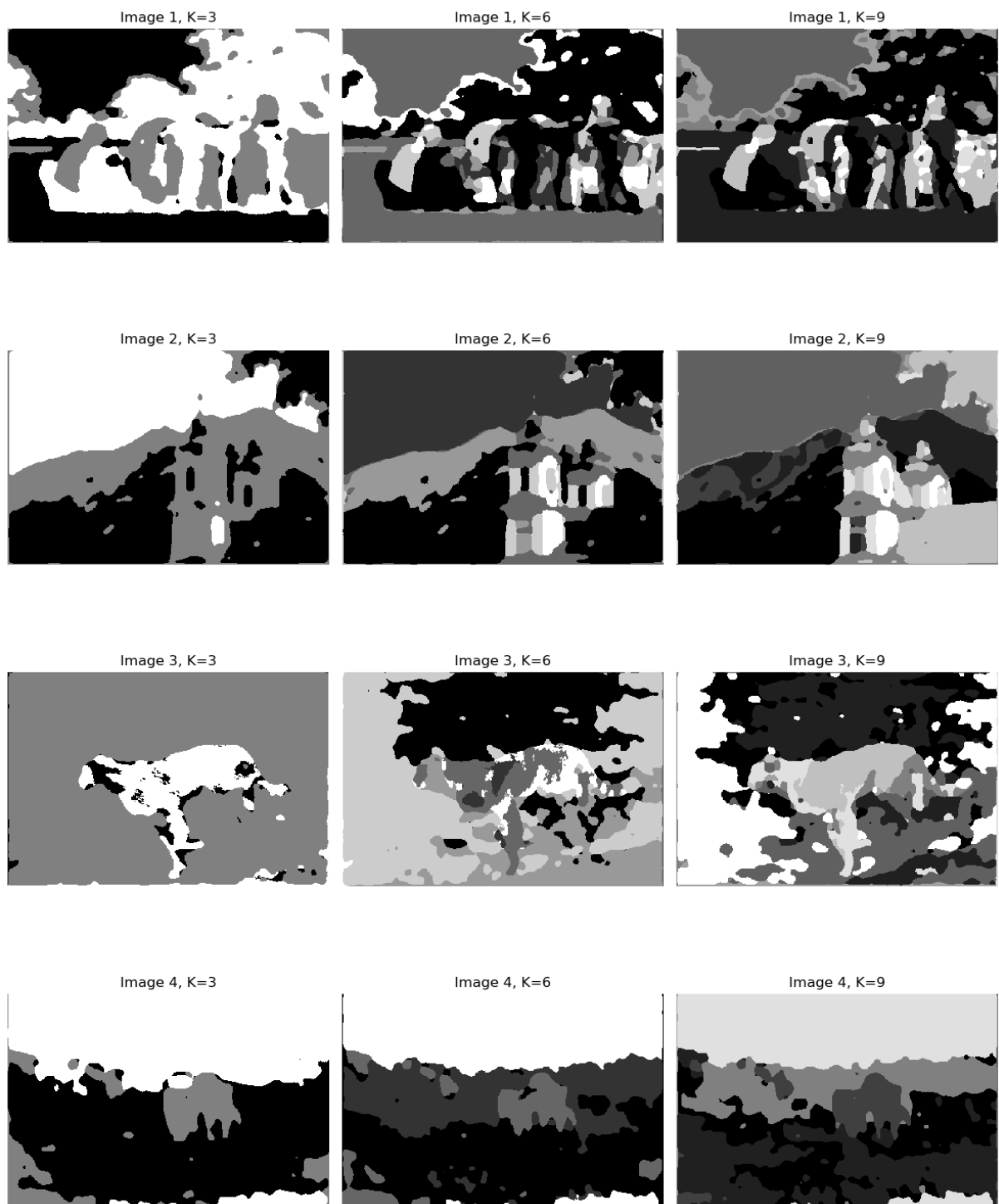


Fig. 3.19 : Le modèle GMM(mélange utilisant des distributions normales sans lissage)

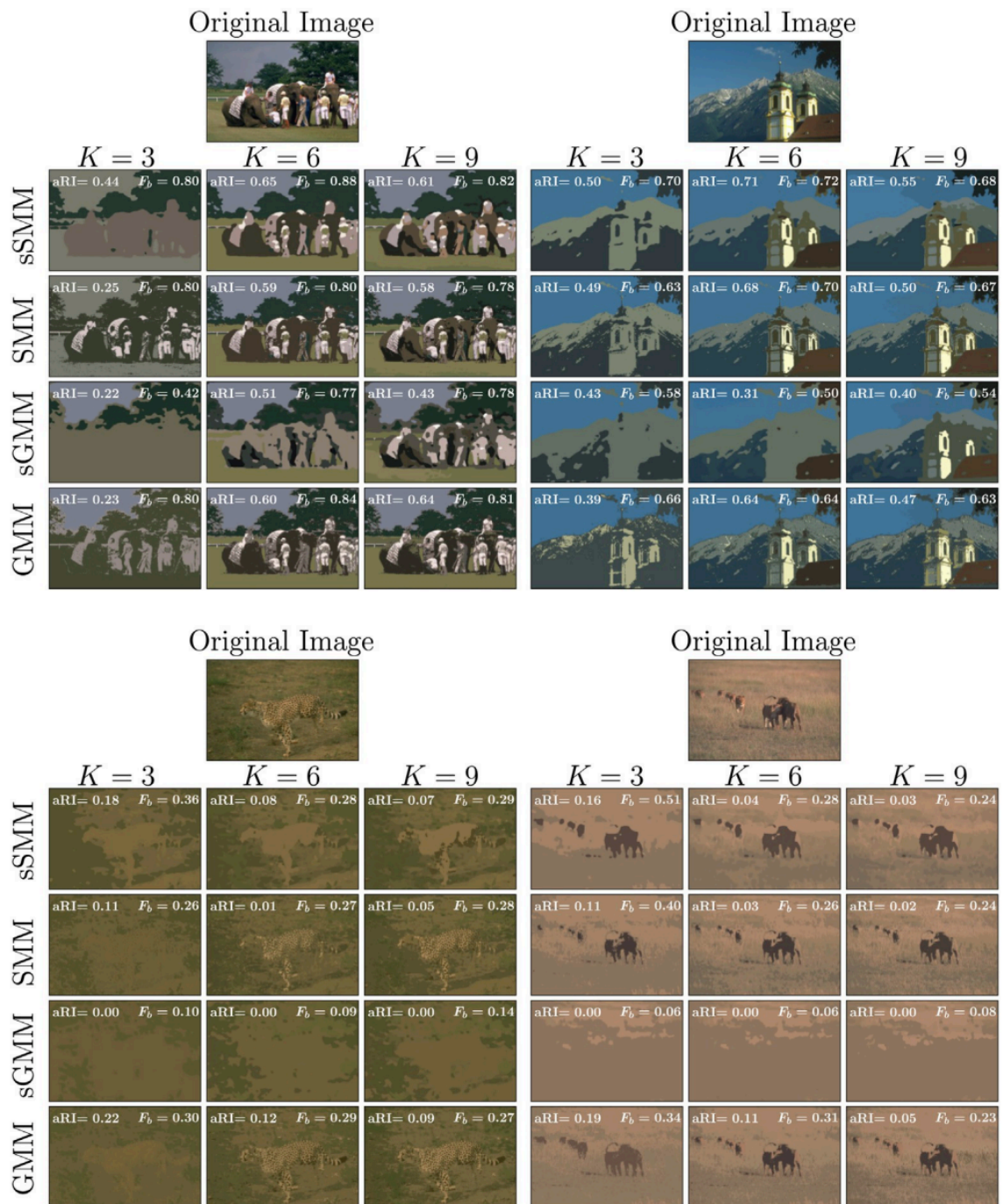


Fig. 3.20 : Résultats obtenus dans l'article

Source : [3]

Résultats article On constate donc que les résultats obtenus coïncident globalement avec ceux rapportés dans l'article de référence : le lissage spatial (sSMM, sGMM) améliore la segmentation, et le modèle sSMM se démarque comme le plus performant et le plus robuste.